

DATA-260 — Homework 4

Xu Wang

Repository: github.com/xuwang-goldenwater/data260-9275

Section 0 — Personal configuration and domain

Value	This submission	How it is derived
SID4	9275	last four digits of the SJSU Student ID
PORT_BASE	8275	$8000 + (9275 \bmod 900)$
PREFIX	s9275	"s" + SID4
SEED	9275	SID4
VERIFY_SEED	269275	$260000 + \text{SID4}$
DOMAIN_ID	3 — grocery supply and recall notices	$9275 \bmod 8$

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB RAM, macOS
Local model	qwen3:8b via Ollama (temperature 0, thinking off); embeddings sentence-transformers/all-MiniLM-L6-v2 on CPU
Database	MySQL 8.0.43, database s9275_rel
Stack	Python 3.12 (conda env data260), FastAPI + SQLAlchemy 2 + PyMySQL; React 18 + Vite + react-router-dom 6
Tagged commit	7024b7f
Tag	hw4

Homework 4 extends the same codebase. The HW3 FastAPI app in `code/auth_app/` becomes the backend for the React client: users and sessions move from in-memory dictionaries into MySQL, and the recall-notice data moves from HW2's JSON file into MySQL. HW2's `code/api_server.py` is unchanged. The React client is in `frontend/`, the Part 3 and Part 4 scripts are in `code/`, and `reports/hw04/` holds evidence only (`RUN_LOG.txt`, `raw/`, `METRICS.md`, `AI_USE.md`, `verification.json`).

Part 1 — React client

Run: `make run-hw04` (API on `http://localhost:8275`) and `make run-frontend` (React on `http://localhost:5173`). Log in with `demo@s9275.example.com / recall-demo-9275`. The entity is a recall notice: primary field `product_name`, secondary field `category`.

1.1 Login (`Login.jsx`) and "Login required"

Login posts email + password to `POST /api/auth/login`. The backend answers with `Set-Cookie`; the component never reads the cookie. When no one is logged in, Home shows "Login required" and the navbar hides "Add Record".

frontend/src/pages/Login.jsx

```

export default function Login({ onLogin }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);

  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    setBusy(true);
    try {
      await onLogin(email.trim(), password);
    } catch (err) {
      setError(err.status === 401 ? "Invalid email or password." : err.message);
      setBusy(false);
    }
  }
}

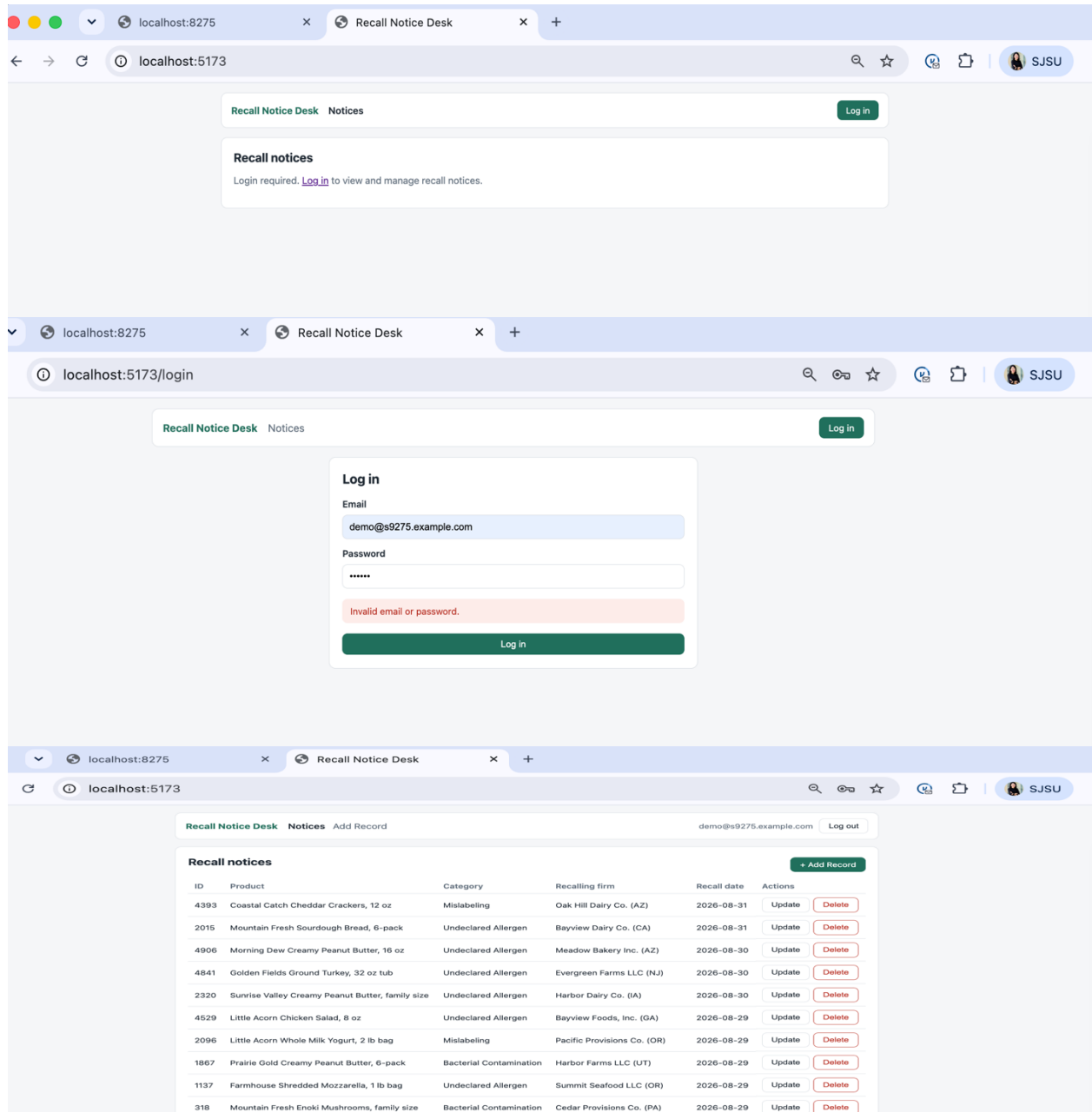
```

frontend/src/components/Navbar.jsx

```

// "Add Record" is only shown to a logged-in user.
export default function Navbar({ user, onLogout }) {
  return (
    <nav className="navbar">
      <Link to="/" className="brand">Recall Notice Desk</Link>
      <div className="nav-links">
        <NavLink to="/" end>Notices</NavLink>
        {user && <NavLink to="/create">Add Record</NavLink>}
      </div>
    </nav>
  );
}

```

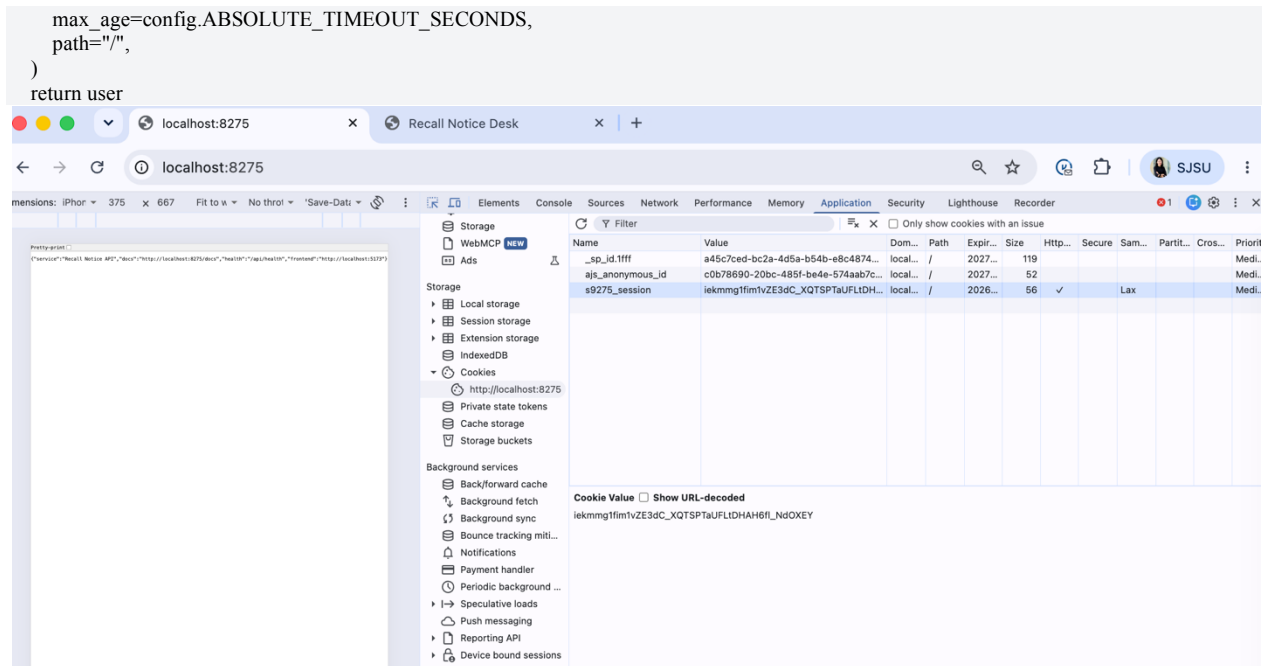


1.2 HTTP-only session cookie

code/auth_app/routers/auth.py — login sets an opaque token in an HTTP-only cookie

```
now = utcnow()
token = secrets.token_urlsafe(32)      # 43 random URL-safe characters
db.add(SessionToken(id=token, user_id=user.id, created_at=now,
                    expires_at=new_expiry(now, now)))
db.commit()

response.set_cookie(
    key=config.COOKIE_NAME,
    value=token,
    httponly=True,      # JavaScript cannot read it
    samesite="lax",     # sent to localhost:8275 from localhost:5173 (same site)
    secure=False,       # plain http on localhost; set True behind HTTPS
```



1.3 Home page (Home.jsx), route /

Home receives the records, loading state and page number from App as props and renders them as a table. The 5,000 seeded rows are shown 50 per page. Update and Delete pass the record id through router state.

frontend/src/pages/Home.jsx

```

export default function Home( { user, notices, loading, page, pageSize, onPageChange } ) {
  const navigate = useNavigate();

  if (!user) {
    return (
      <div className="card">
        <h2>Recall notices</h2>
        <p className="notice">Login required. <Link to="/login">Log in</Link> to view and manage recall notices.</p>
      </div>
    );
  }

  return (
    <div className="card">
      <div className="card-header">
        <h2>Recall notices</h2>
        <Link className="btn primary" to="/create">+ Add Record</Link>
      </div>

```

frontend/src/pages/Home.jsx — row actions

```

      <td className="actions">
        <button className="btn" onClick={() => navigate("/update", { state: { id: n.id } })}>
          Update
        </button>
        <button className="btn danger" onClick={() => navigate("/delete", { state: { id: n.id } })}>
          Delete
        </button>
      </td>

```

localhost:8275

Recall Notice Desk

+

localhost:5173

Recall Notice Desk

Notices

Add Record

demo@8275.example.com

Log out

Recall notices

10

Product

Category

Recalling Item

Recall Date

Actions

4393	Coastal Catch Cheddar Crackers, 12 oz	Mislabeling	Oak Hill Dairy Co. (AZ)	2026-08-31	Update Delete
2015	Mountain Fresh Sourdough Bread, 6-pack	Undeclared Allergen	Bayview Dairy Co. (CA)	2026-08-31	Update Delete
4906	Morning Dew Creamy Peanut Butter, 16 oz	Undeclared Allergen	Meadow Bakery Inc. (AZ)	2026-08-30	Update Delete
4841	Golden Fields Ground Turkey, 32 oz tub	Undeclared Allergen	Evergreen Farms LLC (NJ)	2026-08-30	Update Delete
2320	Sunrise Valley Creamy Peanut Butter, family size	Undeclared Allergen	Harbor Dairy Co. (SA)	2026-08-30	Update Delete
4529	Litex Acorn Chicken Salad, 8 oz	Undeclared Allergen	Bayview Foods, Inc. (SA)	2026-08-29	Update Delete
2096	Litex Acorn Whole Milk Yogurt, 2 lb bag	Mislabeling	Pacific Provisions Co. (OR)	2026-08-29	Update Delete
1867	Prarie Gold Creamy Peanut Butter, 6-pack	Bacterial Contamination	Harbor Farms LLC (UT)	2026-08-29	Update Delete
1187	Farmhouse Shredded Mozzarella, 1 lb bag	Undeclared Allergen	Sunrise Seafood LLC (OR)	2026-08-29	Update Delete
318	Mountain Fresh Enoki Mushrooms, family size	Bacterial Contamination	Cedar Provisions Co. (PA)	2026-08-29	Update Delete
2418	Hutton Point Enoki Mushrooms, 2 lb bag	Bacterial Contamination	Sunrise Mills Inc. (FL)	2026-08-28	Update Delete
1442	Litex Acorn Cheddar Crackers, 2 lb bag	Foreign Material	Blue Ridge Kitchens Inc. (SD)	2026-08-28	Update Delete
1012	Prarie Gold Whole Milk Yogurt, 12 oz	Mislabeling	Heartland Foods, Inc. (OR)	2026-08-27	Update Delete
2944	Hutton Point Pistachio Ice Cream, 12 oz	Mislabeling	Orchard Snacks Co. (FL)	2026-08-26	Update Delete
1929	Blue Heron Chocolate Chip Cookies, 8 oz	Bacterial Contamination	Silver Creek Produce LLC (NY)	2026-08-26	Update Delete
4127	Sunrise Valley Frozen Cheddar Cheese, 16 oz	Foreign Material	Silver Creek Dairy Co. (FL)	2026-08-25	Update Delete
3745	Prarie Gold Pistachio Ice Cream, 16 oz	Foreign Material	Sunrise Seafood LLC (OR)	2026-08-23	Update Delete
2429	Prarie Gold Smoked Salmon, 2 lb bag	Undeclared Allergen	Sunrise Dairy Co. (NC)	2026-08-23	Update Delete
915	Litex Acorn Baby Spinach, 1 lb bag	Undeclared Allergen	Meadow Provisions Co. (NC)	2026-08-23	Update Delete
3249	Market Basket Almond Granola, 16 oz	Undeclared Allergen	Cedar Bakery Inc. (CA)	2026-08-24	Update Delete
3008	Coastal Catch Sourdough Bread, 32 oz tub	Bacterial Contamination	Stone Mill Foods, Inc. (SA)	2026-08-24	Update Delete
1270	Market Basket Cheddar Crackers, 16 oz	Foreign Material	Pine Valley Provisions Co. (CA)	2026-08-24	Update Delete
2989	Mountain Fresh Tahini, 6-pack	Undeclared Allergen	Lakeside Kitchens Inc. (FL)	2026-08-23	Update Delete
4190	Blue Heron Shredded Mozzarella, 16 oz	Undeclared Allergen	Prarie Snacks Co. (TX)	2026-08-22	Update Delete
2722	Coastal Catch Smoked Salmon, 1 lb bag	Mislabeling	Northstar Seafood LLC (MI)	2026-08-22	Update Delete
4546	Farmhouse Fruit Cups, 2 lb bag	Undeclared Allergen	Lakeside Seafood LLC (AZ)	2026-08-21	Update Delete
2191	Green Leaf Chocolate Chip Cookies, 32 oz tub	Undeclared Allergen	Blue Ridge Snacks Co. (IL)	2026-08-21	Update Delete
4048	Prarie Gold Chicken Salad, 12 oz	Undeclared Allergen	Evergreen Dairy Co. (NC)	2026-08-20	Update Delete
4795	Prarie Gold Cheddar Crackers, 12 oz	Undeclared Allergen	Pine Valley Bakery Inc. (WA)	2026-08-19	Update Delete
2943	Sunrise Valley Frozen Raw Shrimp, 8 oz	Undeclared Allergen	Silver Creek Foods, Inc. (MN)	2026-08-17	Update Delete
1219	Coastal Catch Enoki Mushrooms, 12 oz	Mislabeling	Willow Farms LLC (OR)	2026-08-17	Update Delete
1162	Mountain Fresh Sourdough Bread, 1 lb bag	Bacterial Contamination	Red Barn Seafood LLC (NJ)	2026-08-17	Update Delete
238	Heartth & Home Hummus, 1 lb bag	Mislabeling	Heartland Farms LLC (SD)	2026-08-16	Update Delete
261	Sunrise Valley Chicken Salad, family size	Undeclared Allergen	Red Barn Kitchens Inc. (SA)	2026-08-16	Update Delete
4886	Good Grain Almond Granola, 16 oz	Mislabeling	Silver Creek Snacks Co. (CA)	2026-08-14	Update Delete
4798	Good Grain Tahini, 6-pack	Undeclared Allergen	Golden Mills Inc. (IL)	2026-08-13	Update Delete
4701	Market Basket Hummus, 32 oz tub	Bacterial Contamination	Prarie Farms LLC (OH)	2026-08-13	Update Delete
4082	Good Grain Chocolate Chip Cookies, 1 lb bag	Mislabeling	Oak Hill Dairy Co. (AZ)	2026-08-13	Update Delete
2488	Coastal Catch Infant Formula, 2 lb bag	Undeclared Allergen	Sunrise Provisions Co. (IL)	2026-08-13	Update Delete
2385	Farmhouse Hummus, family size	Undeclared Allergen	Red Barn Seafood LLC (NJ)	2026-08-13	Update Delete
2055	Market Basket Protein Bars, family size	Foreign Material	Maple Seafood LLC (FL)	2026-08-13	Update Delete
620	Market Basket Frozen Diced Onions, 6-pack	Foreign Material	Golden Produce LLC (TX)	2026-08-13	Update Delete
16	Market Basket Infant Formula, 32 oz tub	Bacterial Contamination	Sunrise Produce LLC (MN)	2026-08-13	Update Delete
4951	Litex Acorn Whole Milk Yogurt, family size	Mislabeling	Blue Ridge Snacks Co. (IL)	2026-08-12	Update Delete
3892	Golden Fields Hummus, 2 lb bag	Undeclared Allergen	Orchard Dairy Co. (NC)	2026-08-12	Update Delete
1620	True North Infant Formula, 1 lb bag	Bacterial Contamination	Cedar Mills Inc. (OR)	2026-08-10	Update Delete
3794	Litex Acorn Baby Spinach, 8 oz	Undeclared Allergen	Pacific Produce LLC (SA)	2026-08-09	Update Delete
776	Coastal Catch Pistachio Ice Cream, 32 oz tub	Bacterial Contamination	Stone Mill Seafood LLC (TX)	2026-08-08	Update Delete
2796	Litex Acorn Baby Spinach, 12 oz	Undeclared Allergen	Willow Seafood LLC (NY)	2026-08-07	Update Delete
2643	Hutton Point Pistachio Ice Cream, 1 lb bag	Mislabeling	Harbor Foods, Inc. (IL)	2026-08-07	Update Delete

[Previous](#) [Page 1](#) [Next](#)

1.4 Add a new record (CreateRecord.jsx), route /create

frontend/src/pages/CreateRecord.jsx

```
export default function CreateRecord({ onAdd }) {
  const [productName, setProductName] = useState(""); // primary field
  const [category, setCategory] = useState(""); // secondary field
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);
```

```
  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    setBusy(true);
    try {
      await onAdd({ product_name: productName.trim(), category });
    } catch (err) {
      setError(err.message);
    }
    setBusy(false);
  }
```

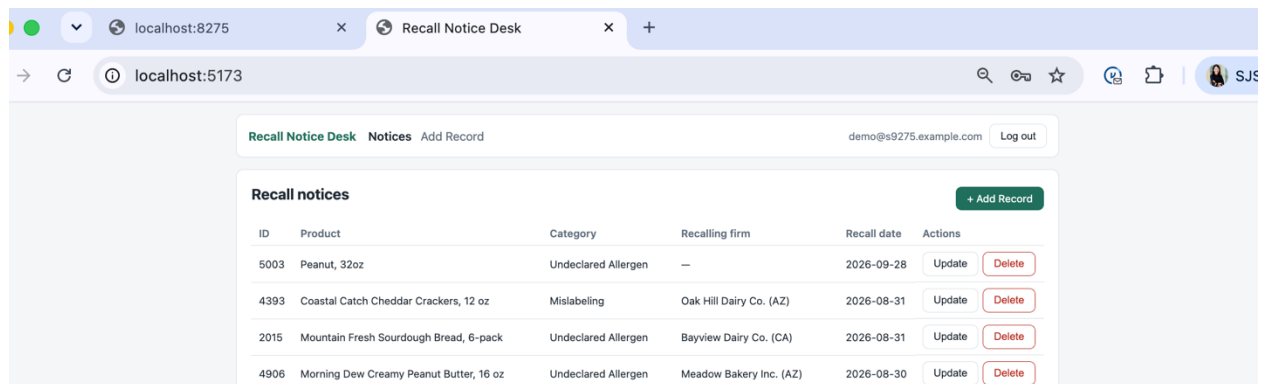
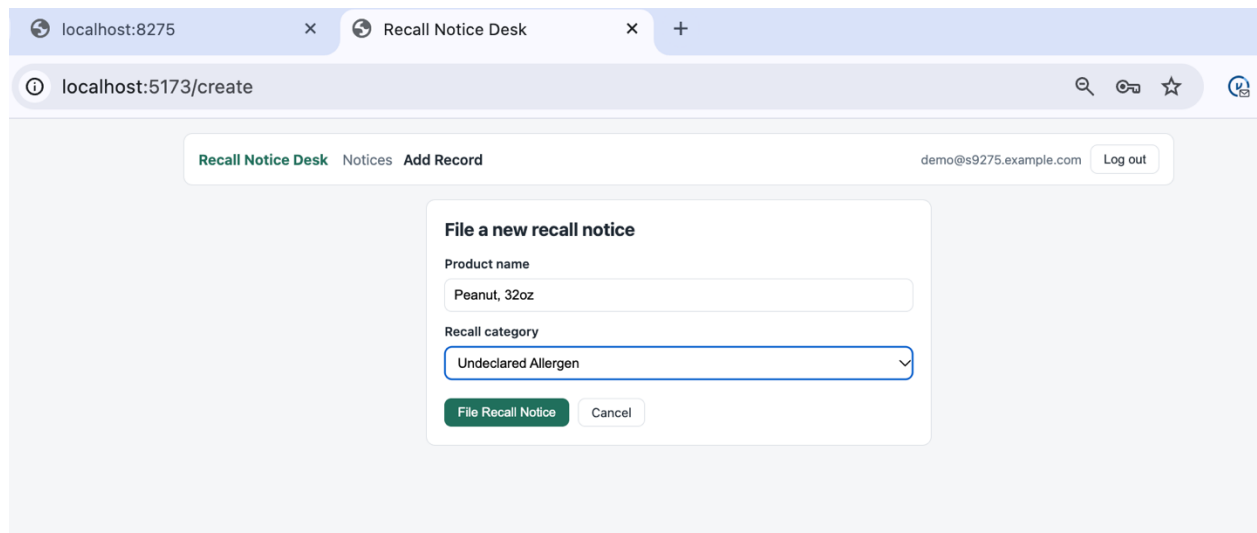
```
}
}
```

frontend/src/pages/CreateRecord.jsx — inputs and submit button

```
<label htmlFor="productName">Product name</label>
<input id="productName" required autoFocus value={productName}
  placeholder="e.g. Sunrise Valley Creamy Peanut Butter, 16 oz"
  onChange={(e) => setProductName(e.target.value)} />

<label htmlFor="category">Recall category</label>
<select id="category" required value={category} onChange={(e) => setCategory(e.target.value)}>
  <option value="" disabled>Choose a category</option>
  {CATEGORIES.map((c) => <option key={c} value={c}>{c}</option>)}
</select>

{error && <div className="alert">{error}</div>}
<div className="form-actions">
  <button className="btn primary" type="submit" disabled={busy}>
    {busy ? "Filing..." : "File Recall Notice"}
  </button>
```



1.5 Update a record (UpdateRecord.jsx), route /update

frontend/src/pages/UpdateRecord.jsx

```

export default function UpdateRecord({ onUpdate }) {
  const id = useLocation().state?.id;
  const [productName, setProductName] = useState("");
  const [category, setCategory] = useState("");
  const [loading, setLoading] = useState(Boolean(id));
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);

  useEffect(() => {
    if (!id) return;
    getNotice(id)
      .then((n) => {
        setProductName(n.product_name);
        setCategory(n.category);
      })
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, [id]);

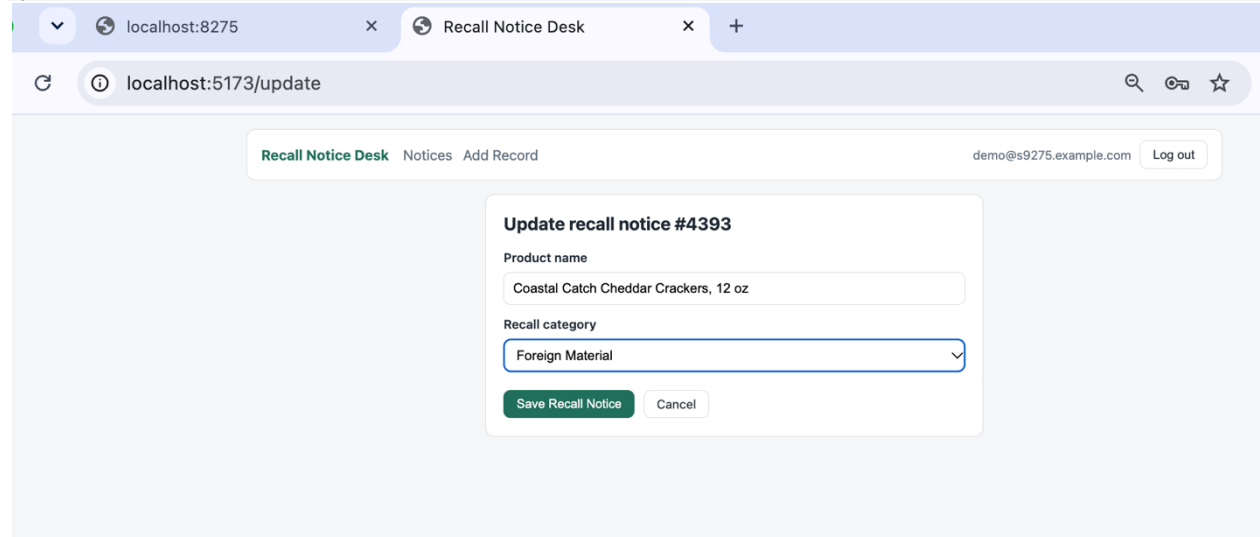
```

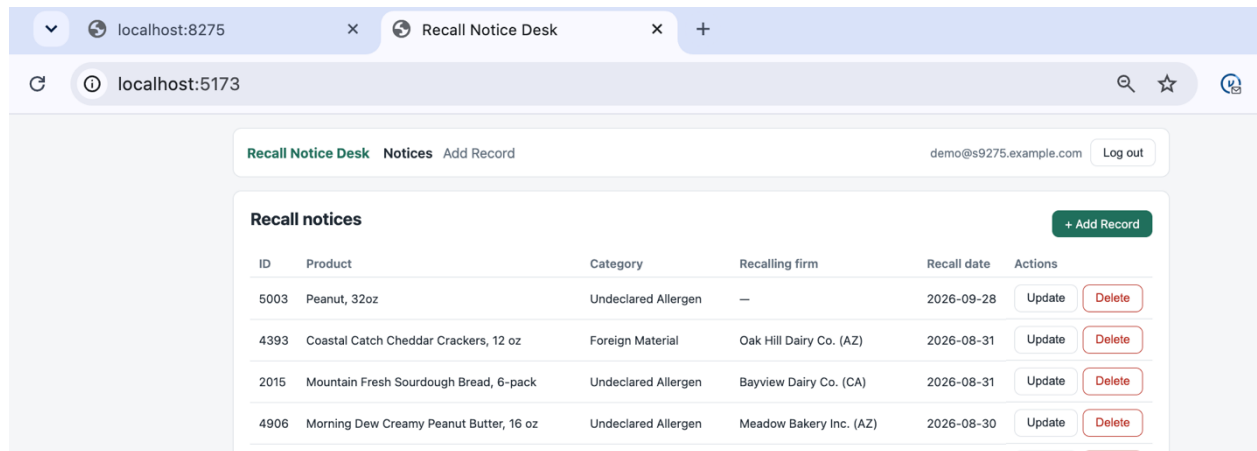
frontend/src/pages/UpdateRecord.jsx — submit

```

async function handleSubmit(e) {
  e.preventDefault();
  setError("");
  setBusy(true);
  try {
    await onUpdate(id, { product_name: productName.trim(), category });
  } catch (err) {
    setError(err.message);
  }
  setBusy(false);
}

```





1.6 Delete a record (DeleteRecord.jsx), route /delete

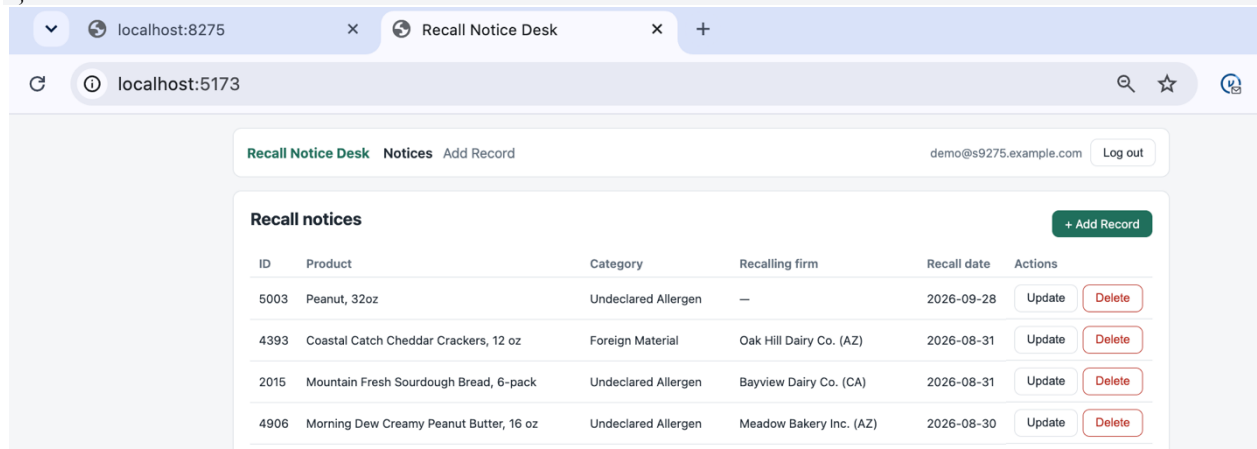
frontend/src/pages/DeleteRecord.jsx

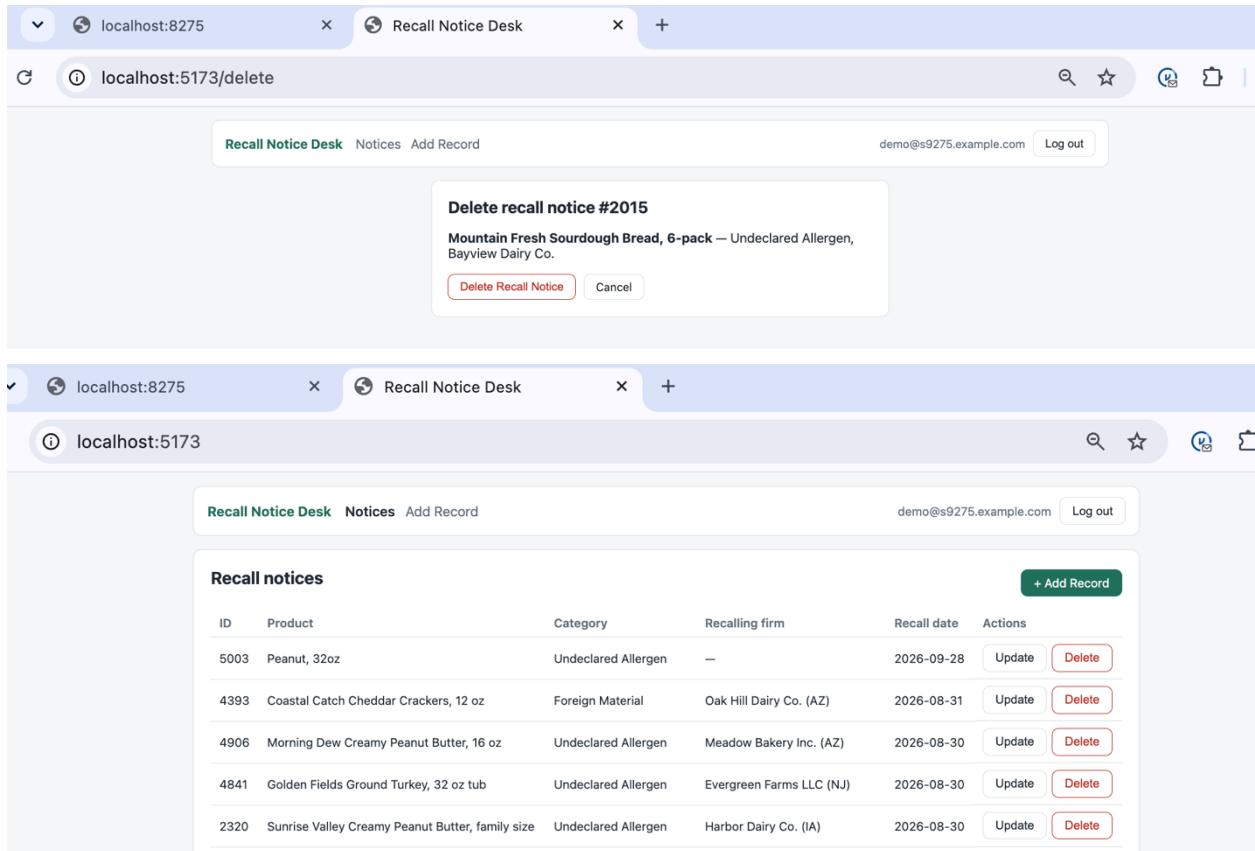
```
export default function DeleteRecord({ onDelete }) {
  const id = useLocation().state?.id;
  const [notice, setNotice] = useState(null);
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);

  useEffect(() => {
    if (!id) return;
    getNotice(id).then(setNotice).catch((err) => setError(err.message));
  }, [id]);
```

frontend/src/pages/DeleteRecord.jsx — button

```
async function handleDelete() {
  setError("");
  setBusy(true);
  try {
    await onDelete(id);
  } catch (err) {
    setError(err.message);
    setBusy(false);
  }
}
```





1.7 Props, hooks and React Router (App.jsx)

App owns the auth state and the list, loads them with `useEffect`, and passes `onAdd` / `onUpdate` / `onDelete` as props. Every handler calls the backend first, so the list always reflects MySQL.

frontend/src/App.jsx — handlers passed as props

```

async function onAdd(notice) {
  await api.createNotice(notice); // MySQL assigns the AUTO_INCREMENT id
  if (page === 1) await loadNotices(); // new notices are dated today -> top of page 1
  else setPage(1);
  navigate("/");
}

async function onUpdate(id, notice) {
  const updated = await api.updateNotice(id, notice);
  setNotices((prev) => prev.map(n => (n.id === id ? updated : n)));
  navigate("/");
}

async function onDelete(id) {
  await api.deleteNotice(id);
  await loadNotices(); // refill the page from the database
  navigate("/");
}

```

frontend/src/App.jsx — routes

```

<Routes>
  <Route
    path="/"
    element={

```

```

    <Home
      user={user}
      notices={notices}
      loading={loading}
      page={page}
      pageSize={PAGE_SIZE}
      onPageChange={setPage}
    />
  }
/>
<Route
  path="/login"
  element={user ? <Navigate to="/" replace /> : <Login onLogin={onLogin} />}
/>
<Route
  path="/create"
  element={<RequireAuth user={user}><CreateRecord onAdd={onAdd} /></RequireAuth>}
/>
<Route
  path="/update"
  element={<RequireAuth user={user}><UpdateRecord onUpdate={onUpdate} /></RequireAuth>}
/>
<Route
  path="/delete"
  element={<RequireAuth user={user}><DeleteRecord onDelete={onDelete} /></RequireAuth>}
/>
<Route path="*" element={<Navigate to="/" replace />} />
</Routes>

```

Part 2 — MySQL persistence and server-side sessions

2.1 Database connection (db_session_based26)

code/auth_app/database.py

```

DATABASE_URL = URL.create(
    "mysql+pymysql",
    username=config.MYSQL_USER,
    password=config.MYSQL_PASSWORD,
    host=config.MYSQL_HOST,
    port=config.MYSQL_PORT,
    database=config.DB_NAME,
    query={"charset": "utf8mb4"},
)

engine = create_engine(DATABASE_URL, pool_pre_ping=True)

# The assignment requires this exact name for the database connection variable.
# Every request gets its own Session from this factory (see get_db).
db_session_based26 = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

Base = declarative_base()

def get_db():
    """FastAPI dependency: one Session per request, always closed afterwards."""
    db = db_session_based26()
    try:
        yield db
    finally:
        db.close()

```

2.2 Schema: s9275_rel

code/sql/hw04_schema.sql is applied by make db-init. users.email is UNIQUE; sessions.id is the session token; recall_notices has an auto-increment id, the primary field product_name and the secondary field category, plus firm_id (Part 3 related table) and recall_date.

code/sql/hw04_schema.sql

```
CREATE TABLE users (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  email   VARCHAR(255) NOT NULL,
  password_hash CHAR(60) NOT NULL,
  CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB;

CREATE TABLE sessions (
  id      VARCHAR(64) PRIMARY KEY,
  user_id INT NOT NULL,
  created_at DATETIME NOT NULL,
  expires_at DATETIME NOT NULL,
  CONSTRAINT fk_sessions_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
) ENGINE=InnoDB;

...

CREATE TABLE recall_notices (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(255) NOT NULL,
  category  VARCHAR(40) NOT NULL,
  firm_id   INT NULL,
  recall_date DATE NOT NULL,
  CONSTRAINT fk_notices_firm FOREIGN KEY (firm_id) REFERENCES firms (id) ON DELETE SET NULL,
  CONSTRAINT ck_notices_category CHECK (category IN
    ('Undeclared Allergen', 'Bacterial Contamination', 'Foreign Material', 'Mislabeling'))
) ENGINE=InnoDB;
```

2.3 Session check on every protected route

The cookie holds only a random token (secrets.token_urlsafef(32), 43 characters). The server looks it up in the sessions table; logout deletes the row, so a copied cookie stops working. Sessions expire after 15 minutes idle and 1 hour absolute (the HW3 rules).

code/auth_app/deps.py

```
def require_session(request: Request, db: Session = Depends(get_db)) -> SessionToken:
    """401 unless the cookie names a live row in the sessions table.

    Costs two SQL statements per request (SELECT the session, UPDATE its
    expiry). Part 3 reports these separately from the list query itself.
    """
    token = request.cookies.get(config.COOKIE_NAME)
    if not token:
        raise HTTPException(status_code=401, detail="Login required")

    row = db.get(SessionToken, token)
    now = utcnow()
    if row is None:
        raise HTTPException(status_code=401, detail="Login required")
    if row.expires_at <= now:
        db.delete(row)
        db.commit()
        raise HTTPException(status_code=401, detail="Session expired")

    row.expires_at = new_expiry(row.created_at, now) # sliding idle timeout
    db.commit()
    return row
```

2.4 CRUD endpoints — Postman

All routes require a session (router-level dependency). The Postman collection is `reports/hw04/postman_collection.json`; run "Login" first so Postman stores the cookie.

code/auth_app/routers/notices.py — create / update / delete

```
@router.post("", response_model=NoticeOut, status_code=201)
def create_notice(payload: NoticeIn, db: Session = Depends(get_db)):
    check_firm(db, payload.firm_id)
    notice = RecallNotice(
        product_name=payload.product_name.strip(),
        category=payload.category,
        recall_date=payload.recall_date or date.today(),
        firm_id=payload.firm_id,
    )
    db.add(notice)
    db.commit()          # MySQL assigns the AUTO_INCREMENT id here
    return load_notice(db, notice.id)

@router.put("/{notice_id}", response_model=NoticeOut)
def update_notice(notice_id: int, payload: NoticeIn, db: Session = Depends(get_db)):
    notice = load_notice(db, notice_id)
    check_firm(db, payload.firm_id)
    notice.product_name = payload.product_name.strip()
    notice.category = payload.category
    if payload.recall_date is not None:
        notice.recall_date = payload.recall_date
    if "firm_id" in payload.model_fields_set: # only change the firm when it was sent
        notice.firm_id = payload.firm_id
    db.commit()
    db.expire(notice)      # reload so the response shows the new firm
    return load_notice(db, notice_id)

@router.delete("/{notice_id}", status_code=204)
def delete_notice(notice_id: int, db: Session = Depends(get_db)):
    notice = load_notice(db, notice_id)
    db.delete(notice)
    db.commit()
    return Response(status_code=204)
```

Operation	Request	Expected
No session	GET /api/notices	401 Login required
Login	POST /api/auth/login	200 + Set-Cookie s9275_session (HttpOnly)
Add (POST)	POST /api/notices	201, new id
View all (GET)	GET /api/notices?page=1&page_size=10	200, 10 items with firm
View by id (GET)	GET /api/notices/{id}	200
Update (PUT)	PUT /api/notices/{id}	200, new values
Delete (DELETE)	DELETE /api/notices/{id}	204; then GET → 404
Logout	POST /api/auth/logout, then GET /api/notices	200, then 401

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

0 Health

GET Health (no login)

1 Auth

GET Login required (no co...

POST Login

GET Me

2 CRUD

POST Create notice (POST)

GET View all notices (GET...

GET View notice by id (GE...

PUT Update notice (PUT)

ENVIRONMENTS

DOCUMENTS

GET Health

GET Login r

POST Login

POST Crea

No Environment

DATA260 HW4 ... > ... > Health (no login)

Save

Share

GET

http://localhost:8275/api/health

Send

Headers

Cookies

Headers

Hide auto-generated headers

Key	Value	Bulk Edit	Presets
✓ Cookie	s9275_session=Lh2T...		
✓ Postman-Token	<calculated when req...		
✓ Host	<calculated when req...		
✓ User-Agent	PostmanRuntime/2.8.0		
✓ Accept	*/*		
✓ Accept-Encoding	gzip, deflate, br		
✓ Connection	keep-alive		
Key	Value	Description	

Body

200

JSON

```
1 {
2   "status": "ok",
3   "database": "s9275_rel",
4   "mysql_version": "8.0.43"
5 }
```

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

0 Health

GET Health (no login)

1 Auth

GET Login required (no co...

POST Login

GET Me

2 CRUD

POST Create notice (POST)

GET View all notices (GET...

GET View notice by id (GE...

PUT Update notice (PUT)

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

GET HealthGET Login rPOST LoginPOST Crea+No Environment

DATA260 HW4 s9275 - Re...>...>LoginSaveShare

POSThttp://localhost:8275/api/auth/loginSend

HeadersCookies

✓	Postman-Token	<calculated when req...	
✓	Content-Type	text/plain	
✓	Content-Length	<calculated when req...	
✓	Host	<calculated when req...	
✓	User-Agent	PostmanRuntime/2.8.0	
✓	Accept	/*/*	
✓	Accept-Encoding	gzip, deflate, br	
✓	Connection	keep-alive	
✓	Content-Type	application/json	
	Key	Value	Description

BodyJSON200{ "id": 1, "name": "Xu Wang", "email": "demo@s9275.example.com" }

The screenshot displays the Postman REST client interface. On the left sidebar, under 'COLLECTIONS', there is a folder '2 CRUD' containing a collection 'POST Create notice (POST)'. The main workspace shows a REST client request configured with the following details:

- Method:** POST
- URL:** http://localhost:8275/api/notices
- Params:** None
- Query Params:** None
- Body:** JSON

The response viewer at the bottom shows a 201 status code and the following JSON body:

```
{
  "id": 5007,
  "product_name": "Harbor Point Frozen Diced Onions, 2 lb bag",
  "category": "Bacterial Contamination",
  "recall_date": "2026-09-28",
  "firm": {
    "id": 12,
    "name": "Cedar Kitchens Inc.",
    "state": "NY"
  }
}
```

The screenshot displays the Postman application interface. On the left sidebar, the 'COLLECTIONS' section is expanded, showing a collection named '2 CRUD'. Under this collection, the request 'GET View all notices (GET...)' is selected. The main workspace shows the details of this request:

- Method:** GET
- URL:** `http://localhost:8275/api/notices?page=1&page_size=10`
- Params:** A table showing query parameters:

Key	Value	Description	Bulk Edit	...
page	1			
page_size	10			
Key	Value	Description		
- Body:** The response body is displayed in JSON format:


```
{
  "version": "fixed",
  "page": 1,
  "page_size": 10,
  "items": [
    {
      "id": 5007,
      "product_name": "Harbor Point Frozen Diced Onions, 2 lb bag",
      "category": "Bacterial Contamination",
      "recall_date": "2026-09-28",
      "firm": {
        "id": 12,

```

The status bar at the bottom indicates a 200 status code and shows the response time and size.

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

GET Me

2 CRUD

POST Create notice (POST)

GET View all notices (GET...)

GET View notice by id (GE...

PUT Update notice (PUT)

DEL Delete notice (DELET...

GET Deleted notice -> 404

3 N+1

GET naive page_size=10

GET fixed page_size=10

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLOWS

POST Create

GET View a

GET View n

PUT Upda

No Environment

DATA26... > ... > View notice by id (GET)

Save

Share

GET

http://localhost:8275/api/notices/{{notice...

Send

Params

Cookies

Query Params

Key	Value	Desc...	Bulk Edit	...
Key	Value	Description		

Body

200

JSON

```
{
  "id": 5007,
  "product_name": "Harbor Point Frozen Diced Onions,
    2 lb bag",
  "category": "Bacterial Contamination",
  "recall_date": "2026-09-28",
  "firm": {
    "id": 12,
    "name": "Cedar Kitchens Inc.",
    "state": "NY"
  }
}
```


Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

GET Me

2 CRUD

POST Create notice (POST)

GET View all notices (GET...

GET View notice by id (GE...

PUT Update notice (PUT)

DEL Delete notice (DELET...

GET Deleted notice -> 404

3 N+1

GET naive page_size=10

GET fixed page_size=10

ENVIRONMENTS

DOCUMENTS

GET fixed pGET naive pGET fixed pGET Deleted

No Environment

DATA260 ... > ... > Deleted notice -> 404

Save

Share

GET

http://localhost:8275/api/notices/{notice...}

Send

Params

Cookies

Query Params

Key	Value	Desc...	Bulk Edit	...
Key	Value	Description		

Body

404

Find a valid recall notice

Diagnose

Decline

{ } JSON

```
1 {
2   "detail": "Recall notice 5007 not found"
3 }
```

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

GET Deleted notice -> 404

3 N+1

- GET naive page_size=10
- GET fixed page_size=10
- GET naive page_size=50
- GET fixed page_size=50
- GET naive page_size=200
- GET fixed page_size=200

4 Logout

- POST Logout
- GET After logout -> 401

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

DATA260 HW4 s9275 - R... > ... > Logout

Save

Share

POST

http://localhost:8275/api/auth/logout

Send

Params

Cookies

Query Params

Key	Value	Desc...	Bulk Edit	...
Key	Value	Description		

Body

200

{ } JSON

1 {

2 "message": "logged out"

3 }

Upgrade files
Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.
Upgrade to v3

COLLECTIONS

- GET Deleted notice -> 404
- 3 N+1
 - GET naive page_size=10
 - GET fixed page_size=10
 - GET naive page_size=50
 - GET fixed page_size=50
 - GET naive page_size=200
 - GET fixed page_size=200
- 4 Logout
 - POST Logout
 - GET After logout -> 401

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLOWS

GET After logout -> 401

Params

Query Params

Key	Value	Desc...	Bulk Edit	...
Key	Value	Description		

Body

401

Authenticate and send request

Fix auth Decline

JSON

```
1 {
2   "detail": "Login required"
3 }
```

main Terminal Console 1 Authenticate and send request

2.5 Database

```
[mysql> show tables;
```

Tables_in_s9275_rel
firms
recall_notices
sessions
users

```
4 rows in set (0.07 sec)
```

```
[mysql> describe users;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
password_hash	char(60)	NO		NULL	

```
4 rows in set (0.06 sec)
```

```
[mysql> describe sessions;
```

Field	Type	Null	Key	Default	Extra
id	varchar(64)	NO	PRI	NULL	
user_id	int	NO	MUL	NULL	
created_at	datetime	NO		NULL	
expires_at	datetime	NO		NULL	

```
4 rows in set (0.01 sec)
```

```
[mysql> describe recall_notices;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
product_name	varchar(255)	NO		NULL	
category	varchar(40)	NO		NULL	
firm_id	int	YES	MUL	NULL	
recall_date	date	NO	MUL	NULL	

```
5 rows in set (0.01 sec)
```

```
mysql> SELECT id, name, email, LEFT(password_hash, 20) FROM users;
```

id	name	email	LEFT(password_hash, 20)
1	Xu Wang	demo@s9275.example.com	\$2b\$12\$P0o5vjPZmygIB

```
1 row in set (0.05 sec)
```

```
mysql> SELECT * FROM sessions;
```

id	user_id	created_at	expires_at
GxBrxKa0qe4yQAFFJ7hQAbJgr5SihdKVNn0ZDjd3xxY	1	2026-09-28 03:52:38	2026-09-28 04:07:41
hhr8RfgpAnT0BAUK3eZNPsp25clr7k5AyYBv_kn9Xbc	1	2026-09-28 03:49:38	2026-09-28 04:04:54
iekmmg1fim1vZE3dC_XQTSPTaUFLtDHAH6fl_Nd0XEY	1	2026-09-28 16:56:14	2026-09-28 17:32:00

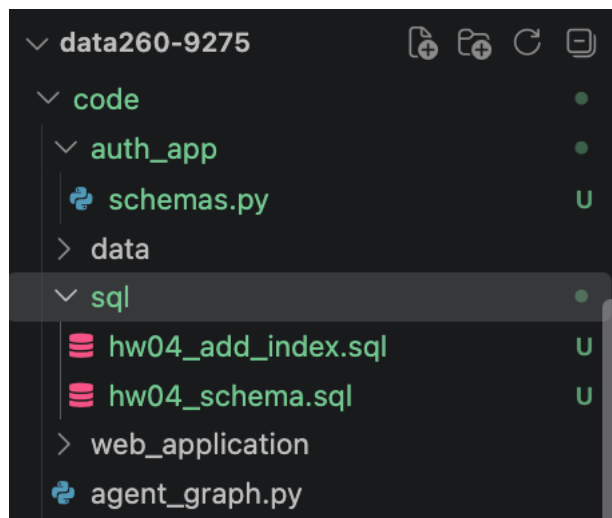
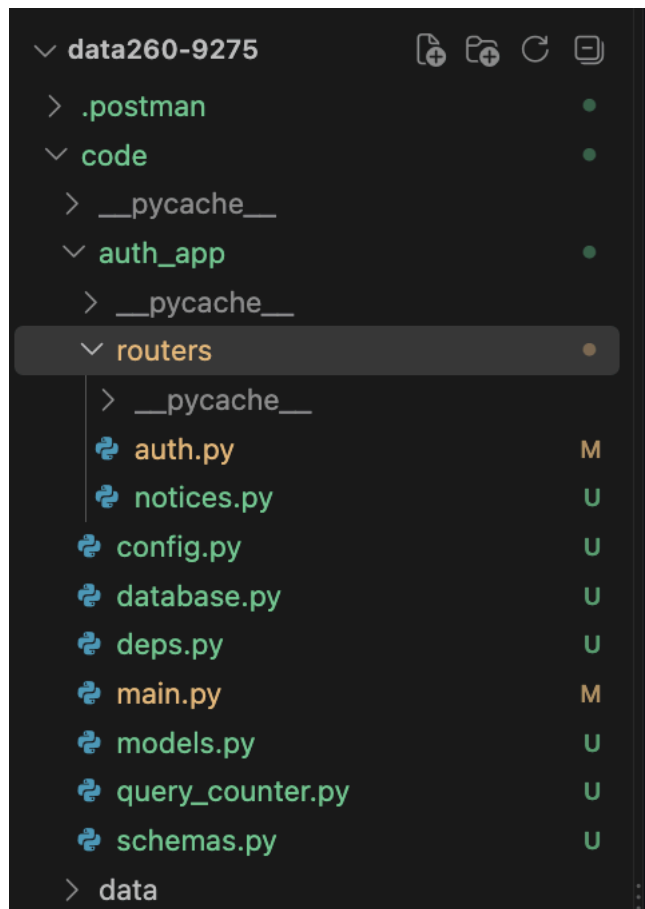
```
3 rows in set (0.01 sec)
```

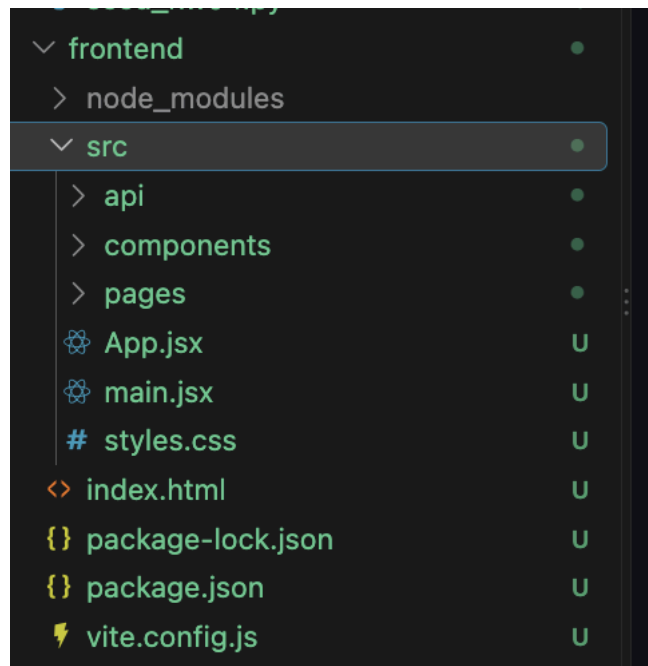
```
mysql> SELECT COUNT(*) FROM recall_notices;
```

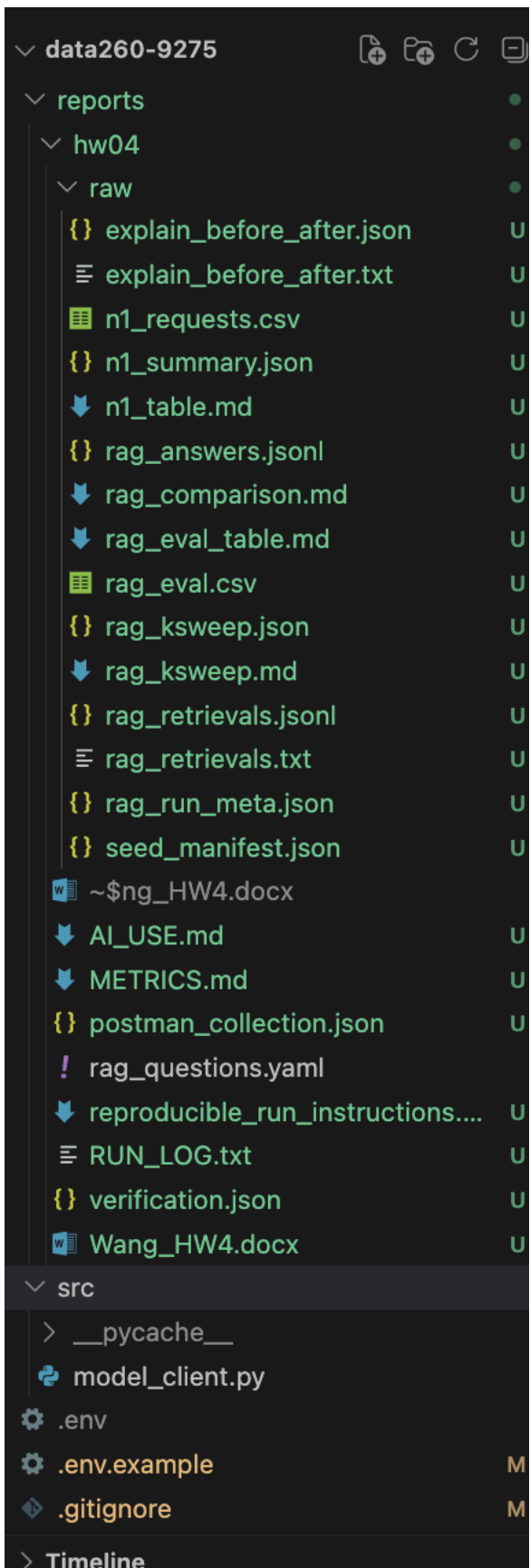
COUNT(*)
5000

```
1 row in set (0.01 sec)
```

2.6 Project folder structure







Part 3 — N+1 measurement and query tuning

3.1 Seed data (SEED 9275)

code/seed_hw04.py creates 200 firms and 5,000 recall notices with random.Random(9275); every notice gets a firm_id in 1..200, so all rows have related data. The SHA-256 of the generated rows (018ba267...) was identical on two machines, so the seed is reproducible. Schema: code/sql/hw04_schema.sql.

code/seed_hw04.py

```
def generate(rng: random.Random):
    firm_names = [f"{w} {t}" for w in FIRM_WORDS for t in FIRM_TYPES] # 240 unique
    rng.shuffle(firm_names)
    firms = [(firm_names[i], rng.choice(STATES)) for i in range(N_FIRMS)]

    categories = [c for c, _ in CATEGORY_WEIGHTS]
    weights = [w for _, w in CATEGORY_WEIGHTS]
    notices = []
    for _ in range(N_NOTICES):
        name = f"{rng.choice(BRANDS)} {rng.choice(PRODUCTS)}, {rng.choice(SIZES)}"
        category = rng.choices(categories, weights)[0]
        firm_id = rng.randint(1, N_FIRMS)
        recall_date = DATE_START + timedelta(days=rng.randint(0, DATE_DAYS))
        notices.append((name, category, firm_id, recall_date.isoformat()))
    return firms, notices
```

```
(data260) Sunny@MacBook-Pro-7 data260-9275 % make seed
===== seed 2026-09-28 11:14:50 =====
python -u code/seed_hw04.py 2>&1 | tee -a reports/hw04/RUN_LOG.txt
{
  "seed": 9275,
  "database": "s9275_rel",
  "firms": 200,
  "recall_notices": 5000,
  "firms_referenced_by_notices": 200,
  "notices_by_category": {
    "Undeclared Allergen": 2078,
    "Mislabeling": 1330,
    "Bacterial Contamination": 801,
    "Foreign Material": 791
  },
  "sha256_of_generated_rows": "018ba267220fdf94d95ec7deb6c67ea989163ce841ee4e68e1a0e6b7835be268",
  "first_notices": [
    [
      "Blue Heron Enoki Mushrooms, 32 oz tub",
      "Undeclared Allergen",
      73,
      "2025-06-23"
    ],
    [
      "Green Leaf Almond Granola, 16 oz",
      "Undeclared Allergen",
      31,
      "2021-02-25"
    ],
    [
      "Sunrise Valley Tahini, 6-pack",
      "Mislabeling",
      173,
      "2023-12-29"
    ]
  ],
  "first_firms": [
    [
      "Willow Snacks Co.",
      "CO"
    ],
    [
      "Prairie Provisions Co.",
      "MN"
    ],
    [
      "Meadow Produce LLC",
      "CA"
    ]
  ],
  "demo_user": "demo@s9275.example.com"
}
```

3.2 Naive (N+1) and fixed (JOIN) list endpoints

Both endpoints return the same page with each notice's firm. The naive version runs one query for the page and one per row; the fixed version uses `joinedload`, which produces a single LEFT OUTER JOIN. A SQLAlchemy `before_cursor_execute` listener counts statements per request and returns them in the X-SQL-Total and X-SQL-List headers.

code/auth_app/routers/notices.py — naive

```
def list_naive(response: Response, page: int = PageNo, page_size: int = PageSize,
               db: Session = Depends(get_db)):
    """N+1 on purpose: one query for the page, then one query per row for its firm.

    db.execute(select(...)) always goes to MySQL. db.get(Firm, id) would not:
    it returns an already-loaded firm from the Session's identity map without
    SQL, which would hide part of the N+1 cost and make the count depend on
    how many firms repeat on the page.
    """
    before = mark_list_start(response)
    notices = db.execute(page_query(page, page_size)).scalars().all()    # 1 query
```

```

items = []
for n in notices:
    # N queries
    firm = None
    if n.firm_id is not None:
        firm = db.execute(select(Firm).where(Firm.id == n.firm_id)).scalar_one()
    items.append(NoticeOut(id=n.id, product_name=n.product_name, category=n.category,
        recall_date=n.recall_date,
        firm=FirmOut.model_validate(firm) if firm else None))
mark_list_end(response, before)
return NoticePage(version="naive", page=page, page_size=page_size, items=items)

```

code/auth_app/routers/notices.py — fixed

```

def fixed_items(db: Session, page: int, page_size: int):
    stmt = page_query(page, page_size).options(joinedload(RecallNotice.firm))
    return db.execute(stmt).scalars().all()    # 1 query: ... LEFT OUTER JOIN firms

@router.get("/fixed", response_model=NoticePage)
def list_fixed(response: Response, page: int = PageNo, page_size: int = PageSize,
    db: Session = Depends(get_db)):
    """Same rows and same JSON as /naive, loaded with a single JOIN."""
    before = mark_list_start(response)
    items = fixed_items(db, page, page_size)
    mark_list_end(response, before)
    return NoticePage(version="fixed", page=page, page_size=page_size, items=items)

```

code/auth_app/query_counter.py

```

@event.listens_for(engine, "before_cursor_execute")
def _count(conn, cursor, statement, parameters, context, executemany):
    counter = _current.get()
    if counter is not None:
        counter.add(statement)

```

3.3 Both versions at each page size — Postman

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

0 Health

GET Health (no login)

1 Auth

GET Login required (no co...)

POST Login

GET Me

2 CRUD

3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLows

DELETE Delete

POST Login

GET naive p

GET Me

No Environment

DATA260 H... > ... > naive page_size=10

Save

Share

GET

http://localhost:8275/api/notices/naive?page_size=10

Send

Headers

Hide auto-generated headers

Key	Value	Bulk Edit	Presets
✓ Cookie	s9275_session=n579...		
✓ Postman-Token	<calculated when req...		
✓ Host	<calculated when req...		
✓ User-Agent	PostmanRuntime/2.8.0		
✓ Accept	*/*		
✓ Accept-Encoding	gzip, deflate, br		
✓ Connection	keep-alive		
Key	Value	Description	

Headers

200

🕒 ⬇ 🌐 🌐

e.g. ...

Key	Value
Date	Mon, 28 Sep 2026 20:06:39 GMT
Server	unicorn
Content-Length	1919
Content-Type	application/json
X-Sql-List	11
X-Server-Time-Ms	676.160
X-Sql-Total	13

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

0 Health

GET Health (no login)

1 Auth

GET Login required (no co...

POST Login

GET Me

2 CRUD

3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLows

OST LogoutGET naive pGET MeGET fixed p+No Environment

DATA260 H...>...>fixed page_size=10SaveShare

GEThttp://localhost:8275/api/notices/fixed?page...Send

ParamsCookies

Query Params

Key	Value	Desc...	Bulk Edit	...
✓page	1			
✓page_size	10			
Key	Value	Description		

Headers200🕒⬇️🌐e.g.⋮

Key	Value
Date	Mon, 28 Sep 2026 20:17:04 GMT
Server	uvicorn
Content-Length	1919
Content-Type	application/json
X-Sql-List	1
X-Server-Time-Ms	445.355
X-Sql-Total	3

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

0 Health

GET Health (no login)

1 Auth

GET Login required (no co...

POST Login

GET Me

2 CRUD

3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLOWS

GET naive pGET MeGET fixed pGET naive p+No Environmentx

DATA260 H... > ... > naive page_size=50SaveShare

GEThttp://localhost:8275/api/notices/naive?p...Send

ParamsCookies

Query Params

Key	Value	Desc...	Bulk Edit	...
☒ page	1			
☒ page_size	50			
Key	Value	Description		

Headers200🕒⬇️🌐e.g.⋮

Key	Value
Date	Mon, 28 Sep 2026 20:17:42 GMT
Server	uvicorn
Content-Length	9343
Content-Type	application/json
X-Sql-List	51
X-Server-Time-Ms	189.836
X-Sql-Total	53

mainTerminalConsole1Ready to go

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

2 CRUD

3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

GET fixed page_size=50

GET naive page_size=200

GET fixed page_size=200

4 Logout

POST Logout

GET After logout -> 401

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLows

GET MeGET fixed pGET naive pGET fixed p+No Environment

DATA260 H...>...>fixed page_size=50SaveShare

GEThttp://localhost:8275/api/notices/fixed?page...Send

ParamsCookies

Query Params

Key	Value	Desc...	Bulk Edit	
☒ page	1		Type	
☒ page_size	50			
Key	Value	Description		

Headers200🕒⬇️🌐🔗⋮

Key	Value
Date	Mon, 28 Sep 2026 20:18:06 GMT
Server	uvicorn
Content-Length	9343
Content-Type	application/json
X-Sql-List	1
X-Server-Time-Ms	123.172
X-Sql-Total	3

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

> 2 CRUD

> 3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

GET fixed page_size=50

GET naive page_size=200

GET fixed page_size=200

> 4 Logout

POST Logout

GET After logout -> 401

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLows

GET naive p

+

No Environment

⋮

DATA260 ... > ... > naive page_size=200

Save

Share

GET

http://localhost:8275/api/notices/naive?p...

Send

Params

Cookies

Query Params

Key	Value	Desc...	Bulk Edit	...
☒ page	1			
☒ page_size	200			
Key	Value	Description		

Headers

200

🕒

📄

🌐

e.g.

...

Key	Value
Date	Mon, 28 Sep 2026 20:18:33 GMT
Server	uvicorn
Content-Length	37030
Content-Type	application/json
X-Sql-List	201
X-Server-Time-Ms	247.547
X-Sql-Total	203

Upgrade files

Postman no longer supports JSON for collections and environments in Local Mode. Upgrade these files to v3 YAML format to continue working with them.

Upgrade to v3

COLLECTIONS

2 CRUD

3 N+1

GET naive page_size=10

GET fixed page_size=10

GET naive page_size=50

GET fixed page_size=50

GET naive page_size=200

GET fixed page_size=200

4 Logout

POST Logout

GET After logout -> 401

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLows

GET naive pGET fixed pGET naive pGET fixed p+No Environment

DATA260 ... > ... > fixed page_size=200SaveShare

GEThttp://localhost:8275/api/notices/fixed?pa...Send

ParamsCookies

Query Params

Key	Value	Desc...	Bulk Edit	
☒ page	1			
☒ page_size	200			
Key	Value	Description		

Headers200🕒⬇️🌐e.g.⋮

Key	Value
Date	Mon, 28 Sep 2026 20:20:20 GMT
Server	uvicorn
Content-Length	37030
Content-Type	application/json
X-Sql-List	1
X-Server-Time-Ms	26.435
X-Sql-Total	3

3.4 Measurements (180 requests)

code/n1_bench.py sends 5 warm-up requests per group, then 30 measured requests per page size and version over HTTP. Naive and fixed are paired on the same random page (seed 9275) and interleaved. Raw data: reports/hw04/raw/n1_requests.csv (180 rows), n1_summary.json. SQL statements are shown as total / list-only; the difference of 2 is the session check (SELECT + UPDATE).

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	13 / 11	10.21	12.33	15.91
10	fixed	3 / 1	9.60	12.94	14.52
50	naive	53 / 51	19.33	20.98	22.33
50	fixed	3 / 1	10.53	12.52	23.29
200	naive	203 / 201	52.51	57.30	67.47
200	fixed	3 / 1	12.76	14.93	15.65

Speed-up (naive p50 / fixed p50): page 10 = 1.06x, page 50 = 1.84x, page 200 = 4.12x. All 180 naive/fixed pairs returned identical items. Measured on 2026-09-28 11:27 with the recall_date index present (it affects both versions equally).

```

[(data260) Sunny@MacBook-Pro-7 data260-9275 % make bench-n1
===== bench-n1 2026-09-28 11:27:34 =====
python -u code/n1_bench.py 2>&1 | tee -a reports/hw04/RUN_LOG.txt
===== bench-n1 2026-09-28T11:27:34 base=http://localhost:8275 reps=30 warmup=5 =====
server: {'status': 'ok', 'database': 's9275_rel', 'mysql_version': '8.0.43'}
rep 1/30 done
rep 2/30 done
rep 3/30 done
rep 4/30 done
rep 5/30 done
rep 6/30 done
rep 7/30 done
rep 8/30 done
rep 9/30 done
rep 10/30 done
rep 11/30 done
rep 12/30 done
rep 13/30 done
rep 14/30 done
rep 15/30 done
rep 16/30 done
rep 17/30 done
rep 18/30 done
rep 19/30 done
rep 20/30 done
rep 21/30 done
rep 22/30 done
rep 23/30 done
rep 24/30 done
rep 25/30 done
rep 26/30 done
rep 27/30 done
rep 28/30 done
rep 29/30 done
rep 30/30 done
| Page size | Version | SQL stmts/req (total / list) | p50 (ms) | p95 (ms) | p99 (ms) |
|---|---|---|---|---|---|
| 10 | naive | 13 / 11 | 10.21 | 12.33 | 15.91 |
| 10 | fixed | 3 / 1 | 9.6 | 12.94 | 14.52 |
| 50 | naive | 53 / 51 | 19.33 | 20.98 | 22.33 |
| 50 | fixed | 3 / 1 | 10.53 | 12.52 | 23.29 |
| 200 | naive | 203 / 201 | 52.51 | 57.3 | 67.47 |
| 200 | fixed | 3 / 1 | 12.76 | 14.93 | 15.65 |

Speed-up (naive p50 / fixed p50): page 10 = 1.06x, page 50 = 1.84x, page 200 = 4.12x
wrote 180 rows to /Users/Sunny/Documents/data260-9275/reports/hw04/raw/n1_requests.csv
[(data260) Sunny@MacBook-Pro-7 data260-9275 % █

```

Why the speed-up grows with page size. The naive endpoint sends $1 + N$ queries, so its time grows linearly with N : from page 10 to page 200 its p50 rises from 10.2 to 52.5 ms, about 0.22 ms per extra row — the cost of one more round trip to MySQL plus building one ORM object. The fixed endpoint sends one JOIN whatever N is, so it stays almost flat ($9.6 \rightarrow 12.8$ ms; the increase is serialising more rows). Both versions pay the same per-request overhead of roughly 9–10 ms (HTTP, the two session statements, FastAPI). At page 10 that overhead dominates and the 10 extra queries add little (1.06x); at page 200 the 200 extra round trips are most of the naive request's time (4.12x). The speed-up is roughly $(\text{overhead} + N \times \text{per-query cost}) / \text{overhead}$, so it keeps growing with N , and it would be much larger with the database on another machine, where each round trip costs milliseconds instead of a fraction of one. With 30 samples p99 is close to the maximum and noisy (e.g. fixed at page 50: p95 12.5 ms but p99 23.3 ms), so p50 is the fair comparison.

3.5 One index and EXPLAIN before / after

code/sql/hw04_add_index.sql

```
CREATE INDEX ix_recall_notices_recall_date ON recall_notices (recall_date);
```

code/explain_index.py drops the index if present, runs EXPLAIN and EXPLAIN ANALYZE on the fixed endpoint's SQL (... ORDER BY recall_date DESC, id DESC LIMIT n) for page 1 at each page size, adds the index, and repeats. Output: reports/hw04/raw/explain_before_after.txt / .json.

Page size	Before: type / key / Extra	After: type / key / Extra	Median ms (20 runs)
10	ALL / - / Using filesort	index / ix_recall_notices_recall_date / Backward index scan	2.38 → 0.37
50	ALL / - / Using filesort	ALL / - / Using filesort	3.42 → 3.14
200	ALL / - / Using filesort	ALL / - / Using filesort	4.82 → 4.92

```

((data260) Sunny@MacBook-Pro-7 data260-9275 % make explain-index
)
===== explain-index 2026-09-28 11:31:15 =====
python -u code/explain_index.py 2>&1 | tee -a reports/hw04/RUN_LOG.txt
===== explain-index 2026-09-28T11:31:16 =====
##### page_size = 10 #####

query:
SELECT recall_notices.id, recall_notices.product_name, recall_notices.category, recall_notices.firm_id, recall_notices.recall_date, firms_1.id AS id_1, firms_1.name, firms_1.state
FROM recall_notices LEFT OUTER JOIN firms AS firms_1 ON firms_1.id = recall_notices.firm_id ORDER BY recall_notices.recall_date DESC, recall_notices.id DESC
LIMIT 0, 10

===== BEFORE (no index on recall_date), page_size 10 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | ALL | None | None | 4970 | 100.0 | Using filesort
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 10 row(s) (cost=2244 rows=10) (actual time=2.54..2.58 rows=10 loops=1)
  -> Nested loop left join (cost=2244 rows=4970) (actual time=2.54..2.58 rows=10 loops=1)
    -> Sort: recall_notices.recall_date DESC, recall_notices.id DESC, limit input to 10 row(s) per chunk (cost=504 rows=4970) (actual time=2.51..2.51 rows=10 loops=1)
    -> Table scan on recall_notices (cost=504 rows=4970) (actual time=0.0285..1.52 rows=5001 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=0.00657..0.00659 rows=1 loops=10)

median of 20 executions: 2.384 ms

===== AFTER (index ix_recall_notices_recall_date), page_size 10 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | index | None | ix_recall_notices_recall_date | 10 | 100.0 | Backward index scan
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 10 row(s) (cost=1243 rows=10) (actual time=0.0915..0.148 rows=10 loops=1)
  -> Nested loop left join (cost=1243 rows=10) (actual time=0.091..0.147 rows=10 loops=1)
    -> Index scan on recall_notices using ix_recall_notices_recall_date (reverse) (cost=0.0171 rows=10) (actual time=0.0869..0.134 rows=10 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=0.00112..0.00113 rows=1 loops=10)

median of 20 executions: 0.373 ms

##### page_size = 50 #####

query:
SELECT recall_notices.id, recall_notices.product_name, recall_notices.category, recall_notices.firm_id, recall_notices.recall_date, firms_1.id AS id_1, firms_1.name, firms_1.state
FROM recall_notices LEFT OUTER JOIN firms AS firms_1 ON firms_1.id = recall_notices.firm_id ORDER BY recall_notices.recall_date DESC, recall_notices.id DESC
LIMIT 0, 50

```

```

===== BEFORE (no index on recall_date), page_size 50 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | ALL | None | None | 4970 | 100.0 | Using filesort
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 50 row(s) (cost=2244 rows=50) (actual time=2.28..2.36 rows=50 loops=1)
  -> Nested loop left join (cost=2244 rows=4970) (actual time=2.28..2.36 rows=50 loops=1)
    -> Sort: recall_notices.recall_date DESC, recall_notices.id DESC, limit input to 50 row(s) per chunk (cost=504 rows=4970) (actual time=2.28..2.28 rows=50 loops=1)
    -> Table scan on recall_notices (cost=504 rows=4970) (actual time=0.0237..1.46 rows=5001 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=0.00141..0.00144 rows=1 loops=50)

median of 20 executions: 3.418 ms

===== AFTER (index ix_recall_notices_recall_date), page_size 50 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | ALL | None | None | 4969 | 100.0 | Using filesort
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 50 row(s) (cost=2244 rows=50) (actual time=2.28..2.33 rows=50 loops=1)
  -> Nested loop left join (cost=2244 rows=4969) (actual time=2.28..2.32 rows=50 loops=1)
    -> Sort: recall_notices.recall_date DESC, recall_notices.id DESC, limit input to 50 row(s) per chunk (cost=504 rows=4969) (actual time=2.28..2.28 rows=50 loops=1)
    -> Table scan on recall_notices (cost=504 rows=4969) (actual time=0.0245..1.47 rows=5001 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=628e-6..653e-6 rows=1 loops=50)

median of 20 executions: 3.142 ms

##### page_size = 200 #####

query:
SELECT recall_notices.id, recall_notices.product_name, recall_notices.category, recall_notices.firm_id, recall_notices.recall_date, firms_1.id AS id_1, firms_1.name, firms_1.state
FROM recall_notices LEFT OUTER JOIN firms AS firms_1 ON firms_1.id = recall_notices.firm_id ORDER BY recall_notices.recall_date DESC, recall_notices.id DESC
LIMIT 0, 200

===== BEFORE (no index on recall_date), page_size 200 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | ALL | None | None | 4970 | 100.0 | Using filesort
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 200 row(s) (cost=2244 rows=200) (actual time=2.48..2.66 rows=200 loops=1)
  -> Nested loop left join (cost=2244 rows=4970) (actual time=2.48..2.65 rows=200 loops=1)
    -> Sort: recall_notices.recall_date DESC, recall_notices.id DESC, limit input to 200 row(s) per chunk (cost=504 rows=4970) (actual time=2.48..2.49 rows=200 loops=1)
    -> Table scan on recall_notices (cost=504 rows=4970) (actual time=0.0228..1.47 rows=5001 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=572e-6..603e-6 rows=1 loops=200)

median of 20 executions: 4.818 ms

===== AFTER (index ix_recall_notices_recall_date), page_size 200 =====
id | select_type | table | type | possible_keys | key | rows | filtered | Extra
1 | SIMPLE | recall_notices | ALL | None | None | 4969 | 100.0 | Using filesort
1 | SIMPLE | firms_1 | eq_ref | PRIMARY | PRIMARY | 1 | 100.0 | None

EXPLAIN ANALYZE:
-> Limit: 200 row(s) (cost=2244 rows=200) (actual time=2.41..2.58 rows=200 loops=1)
  -> Nested loop left join (cost=2244 rows=4969) (actual time=2.41..2.57 rows=200 loops=1)
    -> Sort: recall_notices.recall_date DESC, recall_notices.id DESC, limit input to 200 row(s) per chunk (cost=504 rows=4969) (actual time=2.41..2.42 rows=200 loops=1)
    -> Table scan on recall_notices (cost=504 rows=4969) (actual time=0.023..1.48 rows=5001 loops=1)
    -> Single-row index lookup on firms_1 using PRIMARY (id=recall_notices.firm_id) (cost=0.25 rows=1) (actual time=574e-6..603e-6 rows=1 loops=200)

median of 20 executions: 4.921 ms

##### summary #####
page_size | before: type / key / Extra | after: type / key / Extra | median ms before -> after
10 | ALL / None / Using filesort | index / ix_recall_notices_recall_date / Backward index scan | 2.384 -> 0.373
50 | ALL / None / Using filesort | ALL / None / Using filesort | 3.418 -> 3.142
200 | ALL / None / Using filesort | ALL / None / Using filesort | 4.818 -> 4.921

```

What changed. Without the index, MySQL has no ordered structure on `recall_date`, so for every page it reads the whole table (type=ALL, "Table scan ... rows=5001") and sorts it (Using filesort) just to return the first N rows. With the index, the page-10 plan reads the index backwards and stops after 10 entries (type=index, Backward index scan; estimated rows 4,970 → 10, optimizer cost 2,244 → 1,243). An InnoDB secondary index also stores the primary key, so (`recall_date`, `id`) is already in the requested DESC, DESC order and no sort is needed: the median time fell from 2.38 ms to 0.37 ms (6.4x faster). For page sizes 50 and 200 the optimizer kept the full scan even though the index exists. The index holds only `recall_date` and `id`, so every row found through it needs a second lookup to fetch `product_name` and `category`. On a 5,000-row table that fits in memory, one sequential scan plus an in-memory sort is estimated to be cheaper than 50 or 200 random lookups (the cost stays 2,244 before and after). The plans are identical before and after, so the 3.42 → 3.14 ms and 4.82 → 4.92 ms differences are run-to-run noise, not an effect of the index. On a much larger table, or with a filter such as `WHERE recall_date >= ...` that selects a small range, the index would be chosen for every page size.

Part 4 — Grounded RAG question answering

4.1 Corpus, chunks and index

Corpus: 90 documents reused from HW3 (89 FDA food-recall announcements + 21 CFR Part 7) in reports/hw03/corpus/. SentenceSplitter with chunk_size = 500 and chunk_overlap = 50 tokens gives 182 chunks; each keeps its text, source file and chunk_id. Chunks are embedded with all-MiniLM-L6-v2 on the CPU and stored in a FAISS IndexFlatIP over unit vectors, so the score is cosine similarity.

code/rag.py — chunking

```
def load_chunks():
    """Split every corpus file into chunks: {chunk_id, source, text, tokens}."""
    from llama_index.core.node_parser import SentenceSplitter
    from llama_index.core.utils import get_tokenizer

    tokenizer = get_tokenizer()
    splitter = SentenceSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP)
    chunks = []
    for path in sorted(CORPUS_DIR.glob("*.txt")):
        text = path.read_text(encoding="utf-8")
        for i, piece in enumerate(splitter.split_text(text)):
            chunks.append({
                "chunk_id": f"{path.stem}#{i}",
                "source": path.name,
                "text": piece,
                "tokens": len(tokenizer(piece)),
                "sha": hashlib.sha256(piece.encode("utf-8")).hexdigest()[:16],
            })
    return chunks, tokenizer
```

code/rag.py — FAISS store

```
def __init__(self, chunks, embedder):
    import faiss
    faiss.omp_set_num_threads(1)  # avoid clashing with torch's OpenMP on macOS
    self.chunks = chunks
    self.embedder = embedder
    self.vectors = embedder.encode([c["text"] for c in chunks])
    self.index = faiss.IndexFlatIP(self.vectors.shape[1])
    self.index.add(self.vectors)
    self._query_cache = {}

def query_vector(self, question):
    """Each question is embedded once, so configs B, C and every k see the same vector."""
    if question not in self._query_cache:
        self._query_cache[question] = self.embedder.encode([question])
    return self._query_cache[question]

def search(self, question, k):
    q = self.query_vector(question)
    scores, ids = self.index.search(q, k)
    # FAISS must agree with a plain numpy dot product over all chunks.
    brute = np.argsort(-(self.vectors @ q[0]))[:k]
    if list(ids[0]) != list(brute) and not np.allclose(
        np.sort(self.vectors[ids[0]] @ q[0]), np.sort(self.vectors[brute] @ q[0]), atol=1e-5):
        raise RuntimeError(f"FAISS and numpy disagree for {question!r}: {ids[0]} vs {brute}")
    return [{**self.chunks[i], "row": int(i), "score": round(float(s), 4)}
            for s, i in zip(scores[0], ids[0]) if i >= 0]
```

```

===== run-rag4 2026-09-28 12:31:54 =====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster download

Loading weights: 0%|          | 0/103 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 4727.81it/s]
indexed 182 chunks from 90 documents in 5.4s
embedding self-check: {'device': 'cpu', 'chunk_reencode_max_abs_diff': 0.0, 'query_reencode_max_abs_diff': 0.0}
===== run-rag4 2026-09-28T12:32:15 chunks=182 embed=sentence-transformers/all-MiniLM-L6-v2 llm=qwen3:8b =====

```

4.2 Retrieval printed before the LLM call

For every question and configuration, rag.py prints the retrieved chunks with rank, score, chunk_id and the C decision (kept / dropped and why) before the model is called. Full output: reports/hw04/raw/rag_retrievals.txt and .jsonl.

```

--- main-03 | Q1 | config C | k=3 ---
Q: What toxin caused the recall of a2 Platinum infant formula, and what produces that toxin?
#1 score=0.7382 fda_a2-platinum-usa-label-infant-formula#0 [kept]
  a2 Platinum USA Label Infant Formula Recalled Because of Possible Health Risk Company Announcement Date: May 0...
#2 score=0.5369 fda_nara-organics-recalls-all-lots-nara#0 [kept]
  NARA ORGANICS RECALLS ALL LOTS OF NARA INFANT FORMULA BECAUSE OF POSSIBLE HEALTH RISK Company Announcement Dat...
#3 score=0.5359 fda_a2-platinum-usa-label-infant-formula#1 [kept]
  The total number of units in the three batches is 63,078 of which an estimated 16,428 units were sold to consu...
#4 score=0.5047 fda_nara-organics-recalls-all-lots-nara#2 [dropped: beyond k]
  If your baby has consumed this product and is presenting symptoms of infant botulism including, but not limite...
#5 score=0.5007 fda_nara-organics-recalls-all-lots-nara#1 [dropped: beyond k]
  The lot code can be found on the bottom of each can. - 408125075E14F2 - 708125076E14F2 - 708125083E14F2 - 4081...
#6 score=0.4562 fda_gusto-group-inc-recalls-shrimp-paste#1 [dropped: beyond k]
  (1 LBS)" and "910 G. (2 LBS)," is packaged in a white plastic jar with a red cap. The product is a dark-colore...

```

```

--- main-18 | Q6 | config C | k=3 ---
Q: Who won the 2022 FIFA World Cup?
#1 score=0.1347 fda_mondelez-global-llc-expands-voluntary-recall#1 [dropped: score < 0.35]
  | 4400008667 | N/A | 09MAY2026 10MAY2026 11MAY2026 12MAY2026 7.0Z CA! BAKED BITES BROOKIE 5CT 12 | 44000085650...
#2 score=0.1261 fda_gerber-products-company-announces-voluntary-recall#1 [dropped: score < 0.35]
  27, 2026 Oct. 28, 2026 Oct. 29, 2026 Nov. 20, 2026 Nov. 25, 2026 Nov. 26, 2026 Nov. 26, 2026 Nov. 27, 2026 Dec...
#3 score=0.1121 fda_frito-lay-issues-voluntary-allergy-alert#1 [dropped: score < 0.35]
  - 4:30 p.m. CST, Monday-Friday). If consumers have an allergy or sensitivity to milk, they should not consume ...
#4 score=0.1034 fda_frutas-y-hortalizas-del-sur-sa#0 [dropped: score < 0.35]
  Frutas y Hortalizas Del Sur S.A. Expands Recall to Include One Lot of Great Value Frozen Organic Triple Berry ...
#5 score=0.0939 fda_ambricola-company-issues-recall-cheese-products#1 [dropped: score < 0.35]
  cup | 03/04/26, 03/12/2026 Members Mark 1.5 lb. Bag | 03/25/26, 03/30/26, 04/05/26 Product Sold by Variable Ra...
#6 score=0.0937 fda_mamtakim-inc-recalls-dried-whole-smelts#1 [dropped: score < 0.35]
  The recall was initiated after an FDA inspection and analysis determined that the fish had not been properly e...

```

4.3 Three configurations

- **A — No RAG:** the question alone.
- **B — Basic RAG:** the raw top-3 chunks pasted before the question, no labels, no rules.
- **C — Context-engineered RAG:** 2k candidates → drop score < 0.35 → drop duplicates (same text, or cosine ≥ 0.92 to a kept chunk) → keep the best k → order by score → label [S1]... with source and chunk_id → 1,600-token budget → grounding rules. If nothing survives the threshold, C returns the refusal sentence without calling the model.

code/rag.py — config C context

```

def engineered_context(store, question, k, tokenizer):
    """Config C: filter, de-duplicate, keep k, trim to the token budget.

    Returns (all candidates with a decision each, the kept chunks in order).
    """

```

```

candidates = store.search(question, k * CANDIDATE_MULTIPLIER)
kept, used_tokens = [], 0
for c in candidates:
    if c["score"] < SCORE_THRESHOLD:
        c["decision"] = f"dropped: score < {SCORE_THRESHOLD}"
    elif any(c["sha"] == x["sha"] for x in kept):
        c["decision"] = "dropped: exact duplicate text"
    elif any(store.cosine(c["row"], x["row"]) >= DEDUP_COSINE for x in kept):
        twin = max(kept, key=lambda x: store.cosine(c["row"], x["row"]))
        c["decision"] = f"dropped: near-duplicate of {twin['chunk_id']}"
    elif len(kept) >= k:
        c["decision"] = "dropped: beyond k"
    elif used_tokens + c["tokens"] > CONTEXT_TOKEN_BUDGET:
        c["decision"] = f"dropped: token budget {CONTEXT_TOKEN_BUDGET}"
    else:
        c["decision"] = "kept"
        kept.append(c)
        used_tokens += c["tokens"]
return candidates, kept    # candidates are already in score order

```

code/rag.py — grounding rules

SYSTEM_GROUNDED = f"""You answer questions about food recall notices using ONLY the numbered sources provided.

Rules:

1. Use only facts stated in the sources. Do not use outside knowledge.
2. After every sentence, cite the source number(s) it came from in square brackets, e.g. [S1] or [S1][S3].
3. If the sources do not contain enough evidence to answer, reply with exactly this sentence and nothing else:
{REFUSAL}
4. If the question could refer to several different recalls, say that briefly, then give only what the sources support, with citations.
5. Keep the answer under 120 words."""

4.4 Six questions — results (k = 3)

Questions: reports/hw04/rag_questions.yaml, committed before the retrieval run. Full answers: raw/rag_comparison.md and raw/rag_answers.jsonl.

Q (type)	A — No RAG	B — Basic RAG	C — Context RAG
Q1 one chunk	wrong ("saxitoxin")	correct, grounded	correct, grounded, cites [S1]
Q2 two chunks	wrong ("every 7 days")	correct, not grounded	correct, not grounded
Q3 across documents	incomplete	correct, not grounded	correct, grounded, [S1][S2]
Q4 ambiguous	generic advice	generic advice, not grounded	generic advice, not grounded
Q5 not in documents	hallucinated "\$1.2 million" fine	refused	exact refusal (rules)
Q6 unrelated	answered ("Argentina")	answered wrongly ("France")	exact refusal (threshold, no LLM call)

Q5 (answer not in the documents)

Q: How much was Prairie Farms fined by the FDA for its fat free milk recall?

Expected: Must refuse. The Prairie Farms notice exists, but no document mentions a fine or penalty.

Config	Chunks in prompt (score)	Retrieval ok	Refused	Answer
A	—	None	False	Prairie Farms was fined \$1.2 million by the FDA for its fat-free milk recall in 2013.
B	fda_prairie-farms-announces-recall-prairie-farms#0 (0.78), fda_clover-hill-dairy-recalls-soft-ricottarequeson#1 (0.61), fda_frito-lay-issues-voluntary-allergy-alert#1 (0.58)	None	True	The context provided does not mention any fine imposed by the FDA on Prairie Farms for the fat free milk recall. Therefore, it is not specified how much, if any, Prairie Farms was fined.
C	fda_prairie-farms-announces-recall-prairie-farms#0 (0.78), fda_clover-hill-dairy-recalls-soft-ricottarequeson#1 (0.61), fda_frito-lay-issues-voluntary-allergy-alert#1 (0.58)	None	True	I cannot answer this question from the provided documents.

4.5 top_k sweep (k = 1, 3, 5)

Q	Config	k	Chunks in prompt	Context tokens	Evidence retrieved	Result
Q2	B	1	1	477	no	answered from model knowledge
Q2	B	3	3	1,336	no	definition chunk still missing
Q2	B	5	5	2,304	yes	both chunks in prompt
Q2	C	1	1	477	no	refused
Q2	C	3	3	1,336	no	answered, status part cited
Q2	C	5	3	1,336	no	budget dropped the definition chunk
Q3	B	1	1	495	partial	partial answer
Q3	B	3	3	1,202	yes	complete
Q3	B	5	5	2,157	yes	complete, 2x context
Q3	C	1	1	495	partial	partial answer
Q3	C	3	3	1,202	yes	complete, cited
Q3	C	5	4	1,596	yes	complete; budget dropped 6

More context helped Q2: the Class I definition (§ 7.3) ranks below three other Part 7 chunks and only reached B's prompt at k = 5. It did not help Q3, where k = 3 already held the Listeria paragraph and k = 5 only added more copies of it, nearly doubling the prompt. Irrelevant chunks appeared whenever k exceeded the number of chunks holding the answer (Q2 at k = 3: effectiveness checks § 7.42 and firm-initiated recalls § 7.46). Best k: 3 for Q3 and most questions; Q2 needed 5 in B, and in C the 1,600-token budget cut the needed chunk at k = 5. Raw: raw/rag_ksweep.md / .json.

reports > hnd4 > raw > rag_inspew.md

Modelmist Preview

top_k sweep (k = 1, 3, 5)						
Q	Config	k	Chunks in prompt (score)	Dropped by C	Context tokens	Retrieval ok Auto-correct Answer
Q2	B	1	ecfr_21cfr_part7#11 (0.64)	—	477	False True A Class I recall is a situation where there is a reasonable probability that the use of or exposure to a product will cause serious adverse health consequences ...
Q2	B	3	ecfr_21cfr_part7#11 (0.64), ecfr_21cfr_part7#9 (0.58), ecfr_21cfr_part7#7 (0.57)	—	1336	False True Under 21 CFR Part 7, a Class I recall is the most serious type of recall, involving products that pose a high risk of serious adverse health consequences or dea...
Q2	B	5	ecfr_21cfr_part7#11 (0.64), ecfr_21cfr_part7#9 (0.58), ecfr_21cfr_part7#7 (0.57), ecfr_21cfr_part7#12 (0.57), ecfr_21cfr_part7#1 (0.56)	—	2304	True True Under 21 CFR Part 7, a Class I recall is a situation in which there is a reasonable probability that the use of, or exposure to, a violative product will cause ...
Q2	C	1	ecfr_21cfr_part7#11 (0.64)	ecfr_21cfr_part7#9: beyond k	477	False False I cannot answer this question from the provided documents.
Q2	C	3	ecfr_21cfr_part7#11 (0.64), ecfr_21cfr_part7#9 (0.58), ecfr_21cfr_part7#7 (0.57)	ecfr_21cfr_part7#12: beyond k; ecfr_21cfr_part7#1: beyond k; ecfr_21cfr_part7#4: beyond k	1336	False True A Class I recall is a recall that involves products that can cause serious adverse health consequences or death. The recalling firm generally submits recall sta...
Q2	C	5	ecfr_21cfr_part7#11 (0.64), ecfr_21cfr_part7#9 (0.58), ecfr_21cfr_part7#7 (0.57)	ecfr_21cfr_part7#12: token budget 1600; ecfr_21cfr_part7#1: token budget 1600; ecfr_21cfr_part7#4: token budget 1600; ecfr_21cfr_part7#8: token budget 1600; ecfr_21cfr_part7#10: token budget 1600; ecfr_21cfr_part7#6: token budget 1600; fda_saputo-usa-recalls-great-value-cottage#1: token budget 1600	1336	False True A Class I recall is a situation where there is a reasonable probability that the use of, or exposure to, a violative product will cause serious adverse health c...
Q3	B	1	fda_slade-gorton-co-inc-recalls-one#0 (0.67)	—	495	False False Listeria monocytogenes can cause miscarriages and stillbirths in pregnant women, and can lead to severe symptoms such as high fever, severe headache, stiffness,...
Q3	B	3	fda_slade-gorton-co-inc-recalls-one#0 (0.67), fda_ambrisola-company-issues-recall-cheese-products#0 (0.64), fda_kettle-cuisine-recalls-marketside-tomato-bisque#1 (0.62)	—	1202	True True Listeria monocytogenes can cause serious and sometimes fatal infections in young children, frail or elderly people, and those with weakened immune systems. It c...
Q3	B	5	fda_slade-gorton-co-inc-recalls-one#0 (0.67), fda_ambrisola-company-issues-recall-cheese-products#0 (0.64), fda_kettle-cuisine-recalls-marketside-tomato-bisque#1 (0.62), fda_kettle-cuisine-recalls-marketside-tomato-bisque#0 (0.61), fda_made-fresh-salads-inc-recalls-ready#0 (0.59)	—	2157	True True Listeria monocytogenes can cause serious and sometimes fatal infections in young children, frail or elderly people, and others with weakened immune systems. It ...
Q3	C	1	fda_slade-gorton-co-inc-recalls-one#0 (0.67)	fda_ambrisola-company-issues-recall-cheese-products#0: beyond k	495	False False Listeria monocytogenes can cause short-term symptoms such as high fever, severe headache, stiffness, nausea, abdominal pain, and diarrhea. It can also lead to m...
Q3	C	3	fda_slade-gorton-co-inc-recalls-one#0 (0.67), fda_ambrisola-company-issues-recall-cheese-products#0 (0.64), fda_kettle-cuisine-recalls-marketside-tomato-bisque#1 (0.62)	fda_kettle-cuisine-recalls-marketside-tomato-bisque#0: beyond k; fda_made-fresh-salads-inc-recalls-ready#0: beyond k; fda_nelson-lsa-lacteos-ic-recalls-requeson#1: beyond k	1202	True True Listeria monocytogenes can cause serious and sometimes fatal infections in young children, frail or elderly people, and others with weakened immune systems. It ...
Q3	C	5	fda_slade-gorton-co-inc-recalls-one#0 (0.67), fda_ambrisola-company-issues-recall-cheese-products#0 (0.64), fda_kettle-cuisine-recalls-marketside-tomato-bisque#1 (0.62), fda_qz-produce-lcc-recalls-enoki-mushrooms#0 (0.55)	fda_kettle-cuisine-recalls-marketside-tomato-bisque#0: token budget 1600; fda_made-fresh-salads-icc-recalls-ready#0: token budget 1600; fda_nelson-lsa-lacteos-ic-recalls-requeson#1: token budget 1600; fda_nelson-lsa-lacteos-ic-recalls-requeson#0: token budget 1600; fda_vegmans-recalls-localelli-grated-pecorino-romano#0: token budget 1600; fda_tbc-distribution-corporate-recalls-lucky-foods#f: token budget 1600	1596	True True Listeria monocytogenes can cause serious and sometimes fatal infections in young children, frail or elderly people, and others with weakened immune systems. It ...

4.6 Evaluation

correct_answer and grounded were judged by hand for all 18 rows (raw/rag_eval.csv); retrieval, refusal and format were checked by the script. Accuracy counts a refusal as the correct answer for Q5/Q6. Faithfulness = grounded / answers that were not refusals (small denominators).

Config	Correct retrieval (Q1–Q4)	Accuracy	Faithfulness	Format compliance	Refused Q5/Q6	Wrong refusals Q1–Q4
A — No RAG	n/a	1/6	n/a	n/a	0/2	0/4
B — Basic RAG	3/4	5/6	1/5	n/a	1/2	0/4
C — Context RAG	3/4	6/6	2/4	6/6	2/2	0/4

RAG evaluation (k = 3)

Accuracy = correct answers / 6 (for Q5/Q6 the correct answer is a refusal).

Faithfulness = grounded / answers that were not refusals (B and C only).

Format compliance = C answers that are the exact refusal sentence or cite valid [S#] sources.

Robustness = Q5/Q6 refused, and Q1-Q4 not wrongly refused.

Config	Correct retrieval (Q1-Q4)	Accuracy	Faithfulness	Format compliance	Robustness: refused Q5/Q6	Robustness: over-refusals Q1-Q4
A	n/a	1/6	n/a	n/a	0/2	0/4
B	3/4	5/6	1/5	n/a	1/2	0/4
C	3/4	6/6	2/4	6/6	2/2	0/4

Q	Config	Retrieval	Correct	Grounded	Refused	Format
Q1	A		False	n/a	False	
Q2	A		False	n/a	False	
Q3	A		False	n/a	False	
Q4	A		True	n/a	False	
Q5	A		False	n/a	False	
Q6	A		False	n/a	False	
Q1	B	True	True	yes	False	
Q2	B	False	True	no	False	
Q3	B	True	True	no	False	
Q4	B	True	True	no	False	
Q5	B		True	n/a (refused)	True	
Q6	B		False	no	False	
Q1	C	True	True	yes	False	True
Q2	C	False	True	no	False	True
Q3	C	True	True	yes	False	True
Q4	C	True	True	no	False	True
Q5	C		True	n/a (refused)	True	True
Q6	C		True	n/a (refused)	True	True

4.7 Analysis

Retrieval. Once the embeddings were computed on the CPU (see AI use), MiniLM placed a relevant chunk in the top three for Q1, Q3 and Q4, so correct retrieval was 3/4 for both B and C. Q2 was the hard case: it asks two things, and the status-report section (§ 7.53, score 0.64) outranked the Class I definition (§ 7.3), which only entered the prompt at $k = 5$. For Q5 retrieval did its job—the Prairie Farms notice scored 0.78—but the fact asked for, a fine, is not in it. For Q6 the best chunk scored only 0.13.

Context changes. The score threshold was the most effective single change: it removed every chunk for Q6, so C refused without calling the model, while B passed three unrelated chunks and the model answered "France". Source labels plus grounding rules made C cite [S#] in every answer (format 6/6) and produced the exact refusal for Q5 even though a relevant-looking chunk was present, so the rules, not retrieval, handled Q5. A larger k helped Q2 but not Q3, where $k = 5$ only added more copies of the same Listeria paragraph and doubled the prompt. The token budget only mattered at $k = 5$, and it cut both ways: it held C's context to at most 1,596 tokens where B's grew to 2,157–2,304, but it also dropped the Q2 definition chunk.

De-duplication never fired—the *Listeria* paragraphs are reworded across notices and whole chunks were never 0.92 similar—so sentence-level de-duplication would be needed. For Q4 no configuration said the question was ambiguous; all three merged the similar consumer instructions of several notices.

Hallucination. Without retrieval the model invented a \$1.2 million fine for Q5, answered Q6 from world knowledge, and got Q1 ("saxitoxin") and Q2 ("every 7 days") wrong: accuracy 1/6. B refused Q5 but answered Q6 wrongly; C refused both.

How the pieces combine. Accuracy rose from 1/6 (A) to 5/6 (B) to 6/6 (C), but faithfulness stayed low (B 1/5, C 2/4). For Q2 both B and C were correct without support in their chunks: the model filled in the missing definition from its own knowledge. A correct answer and a grounded answer are therefore different measures. Retrieval quality sets the ceiling—evidence that is not in the window cannot be cited—context engineering decides what the model sees, and the prompt controls citations and refusals. The largest remaining gain would come from retrieval, for example smaller chunks or splitting multi-part questions like Q2, rather than from more prompt rules.

(403 words)

Verification

verify_hw04.py starts the API on PORT_BASE (or reuses a running one) and checks behaviour: React files and routes, schema and db_session_based26, 401 without a session, HttpOnly cookie with an opaque token stored in sessions, a full CRUD round trip checked directly in MySQL, logout invalidating the token, naive = 1 + N vs fixed = 1 list queries with identical data at 10/50/200, 180 raw rows, the index and EXPLAIN files, and the RAG outputs (C refused Q5 and Q6). It writes reports/hw04/verification.json with homework, SID4, commit, tag, model, SEED / VERIFY_SEED and pass/fail per check. It creates one temporary record and one session and removes both; no application file is modified.

```
51 passed, 0 failed -> reports/hw04/verification.json
```

```
(data236) C:\Users\Me\OneDrive\Documents> python verify_hw04.py
```

AI use (AI_USE.md)

1. What I used an AI assistant for, and what I did myself

AI assistant (Claude):

- Wrote the first version of the code after I approved the design: the MySQL-backed API in code/auth_app/, the schema/seed scripts, the React client in frontend/, n1_bench.py, explain_index.py, rag.py, verify_hw04.py, and the Postman collection. It tested these in its own environment against MySQL 8.0 and with a headless browser before handing them over.
- Drafted the six RAG questions.

Me:

- Read the assignment and the instructor's DATA236 demo and listed the gaps between them (the demo logs in with a user_id and no password, protects only the list endpoint, and stores naive datetimes in a time-zone column).

- Made the design decisions: extend the HW3 auth app instead of HW2's `api_server.py` and keep HW2 untouched; put the React client in a root-level frontend/; model the domain as recall notices + firms; pass the record id to `/update` and `/delete` through router state so the routes match the assignment literally.
- Set up and ran everything on my Mac (MySQL 8.0.43, conda env, Ollama qwen3:8b), and fixed the environment problems I ran into (empty MySQL password in `.env`, running in the conda base env with Python 3.13 instead of data260).
- Tested the React app by hand, took the Postman / database / UI screenshots.
- Reviewed the RAG questions and committed them before the retrieval run.
- Read every RAG answer against the retrieved chunks and filled `correct_answer` and grounded in `raw/rag_eval.csv` myself.

2. One AI-produced output that was wrong

The first version of `rag.py` let sentence-transformers choose its device. On my M1 Mac it chose Apple's GPU backend (MPS) for the MiniLM embeddings, while Ollama was using the same GPU for qwen3:8b. The run finished without crashing, but the retrieval was wrong: config B retrieved almost nothing relevant (correct retrieval 1/4) and config C refused all six questions, including Q1–Q4, because every candidate chunk scored below the 0.35 threshold.

3. How I detected the problem

The same question got different retrieval results in different configurations. Config C searches a pool of 2k candidates, so its list must contain B's top 3 with the same scores — but for Q4 the top score was 0.65 in B and 0.11 in C, and for Q1 B's first hit was a lettuce recall while C's was the a2 infant-formula notice. That can only happen if the same question produces different embeddings on different calls. `RUN_LOG.txt` then showed the cause: Metal errors "Insufficient Memory (kIOGPUCommandBufferCallbackErrorOutOfMemory)" on the Apple M1 device during the run. The GPU ran out of memory and the embedding calls silently returned bad vectors.

4. What I changed and why it works now

- Embeddings run on the CPU (`SentenceTransformer(..., device="cpu")`), so they no longer compete with Ollama for GPU memory. MiniLM is small, so the index still builds in seconds.
- Each question is embedded once and cached, so B, C and every k in the sweep use the identical query vector.
- Every FAISS search is cross-checked against a plain numpy dot product over all chunks; a mismatch stops the run.
- Before any model call, `self_check()` re-encodes 8 chunks and all 6 questions and stops the run if the vectors differ at all. The check result and the package versions are saved in `raw/rag_run_meta.json`.

In the re-run the self-check reported a difference of 0.0 for both chunks and queries, B and C show the same ranking and scores for every question (Q4: 0.6502 in both), correct retrieval rose to 3/4 for B and C, and C answered Q1–Q4 and refused only Q5 and Q6. The questions file was not changed between the two runs, so the commit that fixed the questions before retrieval still holds.